

YASH PANDEY

12214802722

CSE - Shift - I (2nd Year - 2023-24)

MAIT ROHINI

DATABASE MANAGEMENT SYSTEMS (DBMS)

(2023-24) SESSION - 4th Sem

(Complete - DBMS - Lab File)

Database Management System LAB
(CIC-256)

Faculty Name : Prof. Namita Gupta

Student Name : Yash Pandey

Roll No : 12214802722 (C-6)

Semester : 4th Semester



Maharaja Agrasen Institute of Technology, PSP Area,
Sector – 22, Rohini, New Delhi - 110086



MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

Computer Science & Engineering Department

VISION

"To be centre of excellence in education, research and technology transfer in the field of computer engineering and promote entrepreneurship and ethical values."

MISSION

To foster an open, multidisciplinary and highly collaborative research environment for producing world-class engineers capable of providing innovative solutions to real-life problems and fulfil societal needs.

Lab Assessment Sheet

S.No	Experiment	Marks					Total Marks	Date	Signature
		R1	R2	R2	R4	R5			
<u>1.</u>	Write all DDL Commands In MySQL					5	10	31/01/24	<i>Cyber</i> 14/2/24
<u>2.</u>	Write all DML Commands In MySQL					5	10	07/02/24	<i>Cyber</i> 14/2/24
<u>3.</u>	GROUP BY, HAVING Clause with 15 Easy LeetCode Ques.					5	10	14/02/24	<i>Cyber</i> 21/2/24
<u>4.</u>	Joins In SQL, 5 Easy & 15 Medium LeetCode Ques.					5	10	21/02/24	<i>Cyber</i> 22/2/24
<u>5.</u>	Design ER-Model, Physical and logical Design and Design							06/03/24	
	The Database from The Given Problem Statement						9		<i>Cyber</i> 13/3/24
<u>5(I).</u>	Check The Highest Normalized form of Each Relation							13/03/24	
	and Take Actions when Required To Normalize The Database								
<u>5(II).</u>	Write SQL and Relational Query on Ques. Based on Project							13/03/24	
<u>6.</u>	Optimize 5 SQL Query Using www.JalbinUse.com							13/03/24	

Lab Assessment Sheet

[illegible]

Lab Exercise-1. (31/01/2024)

Q :- Write All DDL Commands In SQL ?

1. Create Database :- The Create Database Command Is Used To Create A New Database .

SYNTAX ::: CREATE DATABASE DB_Name ;

```
mysql> CREATE DATABASE Yash;
Query OK, 1 row affected (0.02 sec)

mysql> show Databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| yash |
+-----+
5 rows in set (0.00 sec)
```

2. Drop Database :- The Drop Database Command Is Used To Drop An Database .

SYNTAX ::: DROP DATABASE DB_Name ;

```
mysql> DROP DATABASE Yash;
Query OK, 0 rows affected (0.02 sec)

mysql> show Databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.00 sec)
```

3. Create Table :- The Create Table Command Is Used To Create An Table In That Database .

SYNTAX ::: CREATE TABLE Table_name (...);

```
mysql> Use Yash;
Database changed
mysql> CREATE TABLE YASH(EmpID int(5) Primary Key,Emp_Name Varchar(25) Not Null);
Query OK, 0 rows affected, 1 warning (0.07 sec)

mysql> Describe Yash;
```

Field	Type	Null	Key	Default	Extra
EmpID	int	NO	PRI	NULL	
Emp_Name	varchar(25)	NO		NULL	

```
2 rows in set (0.01 sec)
```

4. Drop Table :- The Drop Table Command Is Used To Drop An Table From That Database. The **TRUNCATE TABLE statement** is used to delete the data inside a table, but not the table itself. .

SYNTAX ::: DROP TABLE Table_name ;

```
mysql> DROP TABLE Yash;
Query OK, 0 rows affected (0.07 sec)

mysql> Describe Yash;
ERROR 1146 (42S02): Table 'yash.yash' doesn't exist
```

5. Alter Table :- The Alter Table statement is used to add, delete, or modify columns in an existing table. The ALTER TABLE statement is also used to add and drop various constraints on an existing table.

SYNTAX ::: ALTER TABLE table_name
ADD column_name datatype;

```
mysql> ALTER TABLE Yash ADD DOB Date;  
Query OK, 0 rows affected (0.03 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> Describe Yash;
```

Field	Type	Null	Key	Default	Extra
EmpID	int	NO	PRI	NULL	
Emp_Name	varchar(25)	NO		NULL	
DOB	date	YES		NULL	

3 rows in set (0.00 sec)

6. Constraints :- SQL constraints are Used To Specify Rules For The Data In The Table .

- (i) NOT NULL - Ensures that a column cannot have a NULL value
- (ii) UNIQUE - Ensures that all values in a column are different
- (iii) PRIMARY KEY - A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table.
- (iv) FOREIGN KEY - Prevents actions that Will destroy links b/w tables
- (v) CHECK - Ensures that values in a column satisfies the condition .
- (vi) DEFAULT - Sets a default value for a column if no value is specified
- (vii) CREATE INDEX - Used to create and retrieve data from the database very quickly


```
mysql> CREATE TABLE Yash(EmpID int(5) Primary Key,EmpName Varchar(25) NOT NULL,DOB Date Unique);
Query OK, 0 rows affected, 1 warning (0.11 sec)
```

```
mysql> Describe Yash;
```

Field	Type	Null	Key	Default	Extra
EmpID	int	NO	PRI	NULL	
EmpName	varchar(25)	NO		NULL	
DOB	date	YES	UNI	NULL	

```
3 rows in set (0.00 sec)
```

7.Auto-Increment :- MySQL uses the Auto Increment keyword to perform an auto-increment feature. By default, the starting value for Auto Increment is 1, and it will increment by 1 for each new record. We Can Also Initialize Auto Increment By Any Desired Value .

```
mysql> CREATE TABLE YASH(EmpID INT(5) AUTO_INCREMENT PRIMARY KEY,EmpName VARCHAR(25) NOT NULL,DOB DATE);
Query OK, 0 rows affected, 1 warning (0.04 sec)
```

```
mysql> ALTER TABLE Yash AUTO_INCREMENT=100;
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO Yash(EmpName,DOB) VALUES('Yash Pandey','2003-07-29');
Query OK, 1 row affected (0.04 sec)
```

```
mysql> INSERT INTO Yash(EmpName,DOB) VALUES('Abhishek Pandey','2003-05-17');
Query OK, 1 row affected (0.01 sec)
```

```
mysql> INSERT INTO Yash(EmpName,DOB) VALUES('Aman Pandey','2003-10-23');
Query OK, 1 row affected (0.04 sec)
```

```
mysql> SELECT * From Yash;
```

EmpID	EmpName	DOB
100	Yash Pandey	2003-07-29
101	Abhishek Pandey	2003-05-17
102	Aman Pandey	2003-10-23

```
3 rows in set (0.00 sec)
```

8. Dates :- MySQL comes with the following data types for storing a date or a date/time value in the database :-

- (i) DATE - format -- YYYY-MM-DD
- (ii) DATETIME - format: -- YYYY-MM-DD HH:MI:SS
- (iii) TIMESTAMP - format: -- YYYY-MM-DD HH:MI:SS
- (iv) YEAR - format -- YYYY or YY

9. Create View :- In SQL, a view is a virtual table based on the result-set of an SQL statement. A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database. You can add SQL statements and functions to a view and present the data as if the data were coming from one single table. A view is created with the Create View statement.

SYNTAX ::: CREATE VIEW viewname AS SELECT
col1,col2, ... FROM tablename WHERE condition;

```
CREATE VIEW [Brazil Customers] AS  
SELECT CustomerName, ContactName  
FROM Customers  
WHERE Country = 'Brazil';
```

Lab Exercise-2.

(07/02/2024)

Q :- Write All DML Commands In SQL ?

1. Select :- The Select statement is used to select data from a database. The data returned is stored in a result table, called the result-set.

SYNTAX ::: SELECT * From Table_Name ;

```
mysql> USE Yash;
Database changed
mysql> Select * From Yash;
+-----+-----+-----+
| EmpID | EmpName      | DOB      |
+-----+-----+-----+
| 100   | Yash Pandey  | 2003-07-29 |
| 101   | Abhishek Pandey | 2003-05-17 |
| 102   | Aman Pandey   | 2003-10-23 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

2. Where :- The WHERE clause is used to filter records. It is used to extract only those records that fulfil an specified condition . The Where clause is not only used in Select statements, it is also used in Update, Delete.

SYNTAX ::: SELECT * From Table Where (Conditions);

```
mysql> SELECT * From Yash Where EmpId=100;
+-----+-----+-----+
| EmpID | EmpName      | DOB      |
+-----+-----+-----+
| 100   | Yash Pandey  | 2003-07-29 |
+-----+-----+-----+
1 row in set (0.00 sec)
```


Note -> The Where clause can be combined with AND, OR, and NOT operators.

3. AND Operater :- The AND operator displays a record if all the conditions separated by AND are TRUE.

SYNTAX ::: SELECT * From Table Where (C1 AND C2);

4. OR Operater :- The OR operator displays a record if any Of the conditions separated by OR are TRUE.

SYNTAX ::: SELECT * From Table Where (C1 OR C2);

5. NOT Operater :- The NOT operator displays a record if the condition(s) is NOT TRUE.

SYNTAX ::: SELECT * From Table Where NOT (C1);

```
mysql> Select * From Yash;
```

EmpID	EmpName	DOB
100	Yash Pandey	2003-07-29
101	Abhishek Pandey	2003-05-17
102	Aman Pandey	2003-10-23

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * From Yash Where EmpId=100 AND DOB='2003-07-29';
```

EmpID	EmpName	DOB
100	Yash Pandey	2003-07-29

```
1 row in set (0.00 sec)
```

```
mysql> SELECT * From Yash Where EmpId=100 OR DOB='2003-05-17';
```

EmpID	EmpName	DOB
100	Yash Pandey	2003-07-29
101	Abhishek Pandey	2003-05-17

```
2 rows in set (0.03 sec)
```

```
mysql> SELECT * From Yash Where NOT EmpId=101;
```

EmpID	EmpName	DOB
100	Yash Pandey	2003-07-29
102	Aman Pandey	2003-10-23

```
2 rows in set (0.00 sec)
```

6. ORDER BY :- The Order By keyword is used to sort the result-set in ascending or descending order. Order By keyword sorts the records in Ascending order by default. To sort In Descending order, use the **DESC** keyword.

SYNTAX ::: SELECT column1, column2, ...,
FROM table_name ORDER BY (ASC / DESC);

```
mysql> SELECT * FROM Yash ORDER BY EmpID DESC;
+-----+-----+-----+
| EmpID | EmpName      | DOB      |
+-----+-----+-----+
| 102   | Aman Pandey  | 2003-10-23 |
| 101   | Abhishek Pandey | 2003-05-17 |
| 100   | Yash Pandey   | 2003-07-29 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

7. INSERT :- The INSERT INTO statement is used to insert new records in a table.

SYNTAX ::: INSERT INTO table VALUES (val1 , ...);

```
mysql> INSERT INTO Yash Values(103,"Dev Pandey",'2003-01-14');
Query OK, 1 row affected (0.01 sec)

mysql> Select * From Yash;
+-----+-----+-----+
| EmpID | EmpName      | DOB      |
+-----+-----+-----+
| 100   | Yash Pandey  | 2003-07-29 |
| 101   | Abhishek Pandey | 2003-05-17 |
| 102   | Aman Pandey   | 2003-10-23 |
| 103   | Dev Pandey    | 2003-01-14 |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

8. NULL OPERATER:- A field with a NULL value is a field with no value. If a field in a table is optional, it is possible to insert a new record or update a record without adding a value to this field. Then, the field will be saved with a NULL value. It is not possible to test for NULL values with comparison operators, such as =, <, or <.

SYNTAX ::: SELECT column FROM table WHERE
column_name IS (NULL / NOT NULL);

```
mysql> SELECT * FROM Yash Where EmpID=NULL;
Empty set (0.00 sec)

mysql> SELECT * FROM Yash Where EmpID IS NOT NULL;
+-----+-----+-----+
| EmpID | EmpName          | DOB          |
+-----+-----+-----+
| 100   | Yash Pandey      | 2003-07-29   |
| 101   | Abhishek Pandey  | 2003-05-17   |
| 102   | Aman Pandey      | 2003-10-23   |
| 103   | Dev Pandey       | 2003-01-14   |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

9. UPDATE :- The UPDATE statement is used to modify the existing records in a table.

SYNTAX ::: UPDATE table SET (column =value , ...)
WHERE conditions;

```
mysql> UPDATE YASH SET EmpName="Arun Pandey" Where EmpID=103;
Query OK, 1 row affected (0.03 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> Select * From Yash;
+-----+-----+-----+
| EmpID | EmpName          | DOB          |
+-----+-----+-----+
| 100   | Yash Pandey      | 2003-07-29   |
| 101   | Abhishek Pandey  | 2003-05-17   |
| 102   | Aman Pandey      | 2003-10-23   |
| 103   | Arun Pandey      | 2003-01-14   |
+-----+-----+-----+
4 rows in set (0.00 sec)
```


10. DELETE :- The DELETE statement is used to Delete An Existing records From The table.

SYNTAX ::: DELETE FROM table WHERE conditions ;

```
mysql> DELETE FROM Yash Where EmpID=103;  
Query OK, 1 row affected (0.01 sec)
```

```
mysql> Select * From Yash;
```

EmpID	EmpName	DOB
100	Yash Pandey	2003-07-29
101	Abhishek Pandey	2003-05-17
102	Aman Pandey	2003-10-23

3 rows in set (0.00 sec)

11. LIMIT :- The LIMIT clause is used to **specify the number of records to return**. The LIMIT clause is useful on large tables with thousands of records. Returning a large number of records can impact performance.

SYNTAX ::: SELECT columns FROM table WHERE conditions LIMIT number;

```
mysql> SELECT * FROM Yash LIMIT 2;
```

EmpID	EmpName	DOB
100	Yash Pandey	2003-07-29
101	Abhishek Pandey	2003-05-17

2 rows in set (0.00 sec)

12. MAX & MIN :- The MIN() function returns the smallest value of the selected column. The MAX() function returns the largest value of the selected column.

SYNTAX ::: SELECT MIN(col) FROM table Where conditions;

SYNTAX ::: SELECT MAX(col) FROM table WHERE conditions;

```
mysql> Select * From Yash;
+-----+-----+-----+-----+
| EmpID | EmpName      | DOB       | Salery |
+-----+-----+-----+-----+
| 100   | Yash Pandey  | 2003-07-29 | 50000  |
| 101   | Abhishek Pandey | 2003-05-17 | 75000  |
| 102   | Aman Pandey  | 2003-10-23 | 35000  |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT MAX(Salery) AS LargestSalery FROM Yash;
+-----+
| LargestSalery |
+-----+
| 75000         |
+-----+
1 row in set (0.03 sec)

mysql> SELECT MIN(Salery) AS SmallestSalery FROM Yash;
+-----+
| SmallestSalery |
+-----+
| 35000         |
+-----+
1 row in set (0.00 sec)
```

13. COUNT , AVG & SUM :- The COUNT() returns the number of rows that matches a specified criterion. The AVG() returns the average value of a numeric column. The SUM() returns the total sum of a numeric column.

SYNTAX ::: SELECT COUNT(col) From table Where Condition;

SYNTAX ::: SELECT AVG(col) From table WHERE conditions;

SYNTAX ::: SELECT SUM(col) From table Where conditions;

Here are some examples showing different **LIKE** operators with '%' and '_' wildcards:

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that start with "a"
WHERE CustomerName LIKE '%a'	Finds any values that end with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a_%'	Finds any values that start with "a" and are at least 2 characters in length
WHERE CustomerName LIKE 'a__%'	Finds any values that start with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that start with "a" and ends with "o"

```
mysql> SELECT * FROM Yash Where EmpName LIKE 'a%';
+-----+-----+-----+-----+
| EmpID | EmpName      | DOB       | Salery |
+-----+-----+-----+-----+
| 101   | Abhishek Pandey | 2003-05-17 | 75000 |
| 102   | Aman Pandey    | 2003-10-23 | 35000 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

15. WILDCARDS :- A wildcard character is used to substitute one or more characters in a string.

% -- > Represents zero or more characters

_ --> Represents a single character

```
mysql> SELECT * FROM Yash WHERE EmpName LIKE 'Yas%';
+-----+-----+-----+-----+
| EmpID | EmpName      | DOB       | Salery |
+-----+-----+-----+-----+
| 100   | Yash Pandey  | 2003-07-29 | 50000 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```


16. IN OPERATER :- The IN operator allows you to specify multiple values in a WHERE clause. The IN operator is a shorthand for multiple OR conditions.

SYNTAX ::: SELECT column(s) FROM table WHERE
column IN (value1, value2, ...);

```
mysql> SELECT * FROM Yash WHERE DOB IN('2003-07-29');
+-----+-----+-----+-----+
| EmpID | EmpName      | DOB       | Salary |
+-----+-----+-----+-----+
| 100   | Yash Pandey  | 2003-07-29 | 50000  |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> SELECT * FROM Yash WHERE DOB NOT IN('2003-07-29');
+-----+-----+-----+-----+
| EmpID | EmpName      | DOB       | Salary |
+-----+-----+-----+-----+
| 101   | Abhishek Pandey | 2003-05-17 | 75000  |
| 102   | Aman Pandey    | 2003-10-23 | 35000  |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

17. BETWEEN OPERATER :- The Between operator selects values within a given range. The values can be numbers, text, or dates. The Between operator is inclusive: begin and end values are included.

SYNTAX ::: SELECT column_name(s) FROM table_name
WHERE column_name BETWEEN value1 AND value2;

```
mysql> SELECT * FROM YASH;
+-----+-----+-----+-----+
| EmpID | EmpName      | DOB       | Sallery |
+-----+-----+-----+-----+
| 100   | Yash Pandey  | 2003-07-29 | 50000   |
| 101   | Abhishek Pandey | 2003-05-17 | 75000   |
| 102   | Aman Pandey   | 2003-10-23 | 35000   |
+-----+-----+-----+-----+
3 rows in set (0.01 sec)

mysql> SELECT Sallery From Yash WHERE Sallery BETWEEN 40000 AND 70000;
+-----+
| Sallery |
+-----+
| 50000   |
+-----+
1 row in set (0.00 sec)
```

18. Aliases :- The Aliases are used to give a table, or a column in a table, a temporary name. Aliases are often used to make column names more readable. An alias only exists for the duration of that query. An alias is created with the AS keyword.

SYNTAX ::: SELECT column AS alias FROM table ;

```
mysql> SELECT * FROM YASH;
+-----+-----+-----+-----+
| EmpID | EmpName      | DOB       | Sallery |
+-----+-----+-----+-----+
| 100   | Yash Pandey  | 2003-07-29 | 50000   |
| 101   | Abhishek Pandey | 2003-05-17 | 75000   |
| 102   | Aman Pandey   | 2003-10-23 | 35000   |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT EmpID AS eID FROM Yash;
+-----+
| eID |
+-----+
| 100 |
| 101 |
| 102 |
+-----+
3 rows in set (0.00 sec)
```

Lab Exercise-3. (14/02/2024)

1. Group By CLAUSE :- It is used to group records on the basis of some attribute(s).

SYNTAX ::: SELECT column1, column2, ... FROM
table_name GROUP BY column1, column2, ...;

```
mysql> select * From Yash;
```

EmpID	EmpName	DOB	Salery	Dept
100	Yash Pandey	2003-07-29	50000	FrontEnd
101	Abhishek Pandey	2003-05-17	75000	BackEnd
102	Aman Pandey	2003-10-23	35000	FrontEnd

3 rows in set (0.00 sec)

```
mysql> SELECT Dept AS Department , COUNT(*) AS Employees FROM Yash GROUP BY Dept;
```

Department	Employees
FrontEnd	2
BackEnd	1

2 rows in set (0.01 sec)

2. Having CLAUSE :- It is used to impose some conditions on the groups generated by the group clause.

SYNTAX ::: SELECT column1, column2, ... FROM
table_name GROUP BY column.. Having Conditions;

```
mysql> SELECT Dept AS Department , COUNT(*) AS Employees FROM Yash GROUP BY Dept HAVING COUNT(*)>1;
```

Department	Employees
FrontEnd	2

1 row in set (0.00 sec)

LeetCode Questions.

1. Combine Two Tables (Easy #175)

175. Combine Two Tables Solved ✓

Easy Topics Companies

SQL Schema > Pandas Schema >

Table: Person

Column Name	Type
personId	int
lastName	varchar
firstName	varchar

```
1 SELECT P.firstName, P.lastName, A.city, A.state
2 FROM Person P LEFT JOIN Address A
3 on P.personId = A.personId;
```

Saved to local Ln 3, Col 28

Testcase Test Result

Case 1 +

2. Employee Earning More Than Managers (Easy #181)

181. Employees Earning More Than Their Managers Solved ✓

Easy Topics Companies

SQL Schema > Pandas Schema >

Table: Employee

Column Name	Type
-------------	------

```
1 SELECT T2.name as Employee
2 FROM (Employee AS T1)
3 INNER JOIN (Employee AS T2) ON T1.id = T2.managerID
4 WHERE T1.salary < T2.salary ;
```

Saved to local Ln 1, Col 1

Testcase Test Result

3. QUERY To Track Duplicate Emails (Easy #182)

The screenshot shows a coding platform interface for problem 182, "Duplicate Emails". The problem is marked as "Solved" with a green checkmark. The difficulty is "Easy". The table schema for "Person" is shown as follows:

Column Name	Type
id	int
email	varchar

The SQL query provided in the code editor is:

```
1 # Write your MySQL query statement below
2 -- SELECT T2.email FROM (Person AS T1) , (Person AS T2) WHERE T1.
3 select Email from Person
4 Group by Email
5 Having Count(*) > 1 ;
```

The interface also shows tabs for "Description", "Editorial", "Solutions", and "Submissions". The "Code" tab is active, showing the MySQL query. The "Testcase" tab is also visible, showing "Case 1".

4. Second Highest Salary (Easy # 176)

The screenshot shows a coding platform interface for problem 176, "Second Highest Salary". The problem is marked as "Solved" with a green checkmark. The difficulty is "Medium". The table schema for "Employee" is shown as follows:

Column Name	Type
id	int
salary	int

The SQL query provided in the code editor is:

```
1 # Write your MySQL query statement below
2 SELECT MAX(salary) AS SecondHighestSalary FROM Employee WHERE
3 salary < (SELECT MAX(salary) FROM Employee );
```

The interface also shows tabs for "Description", "Editorial", "Solutions", and "Submissions". The "Code" tab is active, showing the MySQL query. The "Testcase" tab is also visible, showing "Case 1" and "Case 2".

5. Customer Placing The Largest Order (Easy #586)

The screenshot shows the LeetCode interface for problem 586. The problem is marked as 'Solved'. The description on the left includes a table schema for the 'Orders' table. The code editor on the right contains a MySQL query that selects the customer number with the highest order count.

586. Customer Placing the Largest Number of Orders Solved ✓

Easy Topics Companies Hint

SQL Schema > Pandas Schema >

Table: Orders

Column Name	Type
order_number	int
customer_number	int

Code

```
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT customer_number FROM Orders GROUP BY
3 customer_number ORDER BY COUNT(*) DESC LIMIT 1
4
5
```

Restored from local Upgrade to Cloud Saving Ln 1, Col 1

Testcase Test Result

Case 1 +

6. Customers Who Never Orders (Easy #183)

The screenshot shows the LeetCode interface for problem 183. The problem is marked as 'Solved'. The description on the left includes a table schema for the 'Customers' table. The code editor on the right contains a MySQL query that uses a LEFT JOIN to find customers who have no orders.

183. Customers Who Never Order Solved ✓

Easy Topics Companies

SQL Schema > Pandas Schema >

Table: Customers

Column Name	Type
id	int
name	varchar

Code

```
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT C.name AS Customers FROM (Customers AS C)
3 LEFT JOIN (Orders AS O) ON
4 C.id = O.customerId WHERE O.customerId IS NULL;
```

Saved to local Ln 4, Col 48

Testcase Test Result

Accepted Runtime: 196 ms

7. Rising Temperature Problem (Easy #197)

The screenshot shows the LeetCode interface for problem 197, "Rising Temperature". The problem is marked as "Solved" with a green checkmark. The difficulty is "Easy". The table "Weather" is defined with columns: id (int), recordDate (date), and temperature (int). The SQL query in the code editor is:

```
1 # Write your MySQL query statement below
2 SELECT W1.id FROM (Weather AS W1) , (Weather AS W2) WHERE
3 datediff(W1.recordDate, W2.recordDate) = 1
4 AND W1.temperature > W2.temperature;
```

The test result shows "Accepted" with a runtime of 110 ms.

8. Big Countries Problem (Easy #595)

The screenshot shows the LeetCode interface for problem 595, "Big Countries". The problem is marked as "Solved" with a green checkmark. The difficulty is "Easy". The table "World" is defined with columns: name (varchar), continent (varchar), and area (int). The SQL query in the code editor is:

```
1 # Write your MySQL query statement below
2 SELECT name , population , area FROM World WHERE
3 area >= 3000000 OR population >=25000000
```

The test result shows "Accepted" with a runtime of 110 ms.

9. Class More Than 5 Students (Easy #596)

The screenshot shows the LeetCode problem page for '596. Classes More Than 5 Students'. The problem is marked as 'Solved' with a green checkmark. The difficulty is 'Easy'. The table 'Courses' is defined with columns 'student' (varchar) and 'class' (varchar). The code editor shows a MySQL query: `SELECT class FROM Courses GROUP BY class HAVING COUNT(*)>=5;`. The interface includes tabs for Description, Editorial, Solutions, and Submissions, along with buttons for Run, Submit, and a Premium badge.

596. Classes More Than 5 Students Solved

Easy Topics Companies

SQL Schema > Pandas Schema >

Table: Courses

Column Name	Type
student	varchar
class	varchar

Code

```
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT class FROM Courses GROUP BY class HAVING COUNT(*)>=5;
```

Saved to local Ln 1, Col 1

Testcase Test Result

You must run your code first

10. Game Play Analysis-I (Easy #511)

The screenshot shows the LeetCode problem page for '511. Game Play Analysis I'. The problem is marked as 'Solved' with a green checkmark. The difficulty is 'Easy'. The table 'Activity' is defined with columns 'player_id' (int), 'device_id' (int), 'event_date' (date), and 'games_played' (int). The code editor shows a MySQL query: `SELECT player_id , MIN(event_date) AS first_login FROM Activity GROUP BY player_id;`. The interface includes tabs for Description, Editorial, Solutions, and Submissions, along with buttons for Run, Submit, and a Premium badge.

511. Game Play Analysis I Solved

Easy Topics Companies

SQL Schema > Pandas Schema >

Table: Activity

Column Name	Type
player_id	int
device_id	int
event_date	date
games_played	int

Code

```
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT player_id , MIN(event_date) AS first_login FROM
3 Activity GROUP BY player_id;
```

Restored from local Upgrade to Cloud Saving Ln 3, Col 1

Testcase Test Result

You must run your code first

11. Find Customer Referral (Easy #584)

The screenshot shows the LeetCode interface for problem 584. The left sidebar contains the problem description, which includes a table schema for the 'Customer' table. The main area shows the problem title '584. Find Customer Referee' with a 'Solved' status. The right sidebar contains a code editor with a MySQL query.

Table: Customer

Column Name	Type
id	int
name	varchar
referee_id	int

Code

```
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT name FROM Customer WHERE referee_id <> 2
3 OR referee_id IS null;
```

Restored from local Upgrade to Cloud Saving Ln 3, Col 4

Testcase Test Result

You must run your code first

12. Employee Bonus Problem (Easy #577)

The screenshot shows the LeetCode interface for problem 577. The left sidebar contains the problem description, which includes a table schema for the 'Employee' table. The main area shows the problem title '577. Employee Bonus' with a 'Solved' status. The right sidebar contains a code editor with a MySQL query.

Table: Employee

Column Name	Type
empId	int
name	varchar
supervisor	int
salary	int

Code

```
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT E.name , B.bonus FROM (Employee AS E)
3 LEFT JOIN (Bonus AS B) ON E.empId=B.empId
4 WHERE B.bonus < 1000 OR B.bonus IS null;
```

Saved to local Ln 4, Col 1

Testcase Test Result

You must run your code first

13. Sales Problem (Easy #607)

The screenshot shows the LeetCode interface for problem 607, 'Sales Person'. The problem is marked as 'Solved' with a green checkmark. The difficulty is 'Easy'. The table 'SalesPerson' is defined with columns: sales_id (int), name (varchar), and salary (int). The MySQL query editor shows a query that selects the names of salespersons who have not made any orders, specifically filtering for those in the 'RED' company.

607. Sales Person Solved ✓

Easy Topics Companies Hint

SQL Schema > Pandas Schema >

Table: SalesPerson

Column Name	Type
sales_id	int
name	varchar
salary	int

Code

```
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT SalesPerson.name FROM SalesPerson WHERE
3 SalesPerson.name NOT IN
4 ( SELECT SP.name FROM (SalesPerson AS SP) ,
5   (Company AS C) , (Orders AS O) WHERE
6   C.name="RED" AND C.com_id = O.com_id AND
7   SP.sales_id = O.sales_id );
8
```

Restored from local Upgrade to Cloud Saving Ln 7, Col 33

Testcase Test Result

14. Biggest Single Number (Easy #619)

The screenshot shows the LeetCode interface for problem 619, 'Biggest Single Number'. The problem is marked as 'Solved' with a green checkmark. The difficulty is 'Easy'. The table 'MyNumbers' is defined with a column: num (int). The MySQL query editor shows a query that selects the maximum value of 'num' from 'MyNumbers', but only if it appears exactly once.

619. Biggest Single Number Solved ✓

Easy Topics Companies

SQL Schema > Pandas Schema >

Table: MyNumbers

Column Name	Type
num	int

Code

```
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT MAX(num) AS num FROM
3 (SELECT num FROM MyNumbers GROUP BY num
4  HAVING COUNT(*)=1) AS Yash;
```

Saved to local Ln 4, Col 1

Testcase Test Result

15. Not Boring Movies (Easy #620)

 Problem List < > 🔍

Run Submit ⌚ 📄

🔧 ⚙️ 🔥 0 👤 Premium

Description Editorial Solutions Submissions

620. Not Boring Movies Solved ✓

Easy Topics Companies

SQL Schema > Pandas Schema >

Table: Cinema

Column Name	Type
id	int
movie	varchar
description	varchar
rating	float

</> Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 SELECT id , movie , description , rating FROM Cinema
3 WHERE MOD(id,2) <> 0 AND description<>"boring"
4 ORDER By rating DESC ;
```

Restored from local Upgrade to Cloud Saving Ln 4, Col 1

☒ Testcase > Test Result

You must run your code first

LeetCode Profile Page



Yash Pandey
pandeyyash041

Rank 2,142,995

college Sophomore At MAIT CSE ,
Preparing for Top MNC , Learning Java with
DSA And Curious About Web Development

Edit Profile

Solved Problems

21
Solved

Easy	20 / 771	Beats 52.5%
Medium	1 / 1599	Not enough data
Hard	0 / 675	Not enough data

42 submissions in the last year ⓘ

Languages

MySQL 19 problems solved

Java 2 problems solved

Skills

• **Advanced**
Not enough data

• **Intermediate**
Database x19 Bit Manipulation x1

• **Fundamental**
Array x2 Sorting x1
Two Pointers x1

Lab Exercise-4. (21/02/2024)

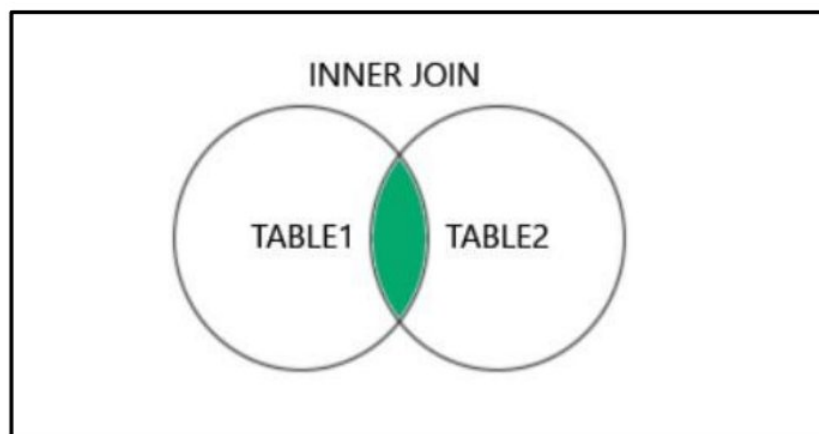
Q :- Write All Joins In SQL ?

An JOIN clause In SQL is used to combine rows from two or more tables, based on a related column between them.

- **INNER JOIN** : Returns records that have matching values in both tables .
- **LEFT JOIN** : Returns all records from the left table, and the matched records from the right table .
- **RIGHT JOIN** : Returns all records from the right table, and the matched records from the left table .
- **CROSS JOIN** : Returns all records from both tables .

1. Inner Join :- The INNER JOIN keyword selects records that have matching values in both tables..

SYNTAX ::: SELECT columns FROM table1 INNER JOIN table2 ON table1.column = table2.column;



```
mysql> SELECT * FROM Yash;
```

EmpID	EmpName	DOB	Location
100	Yash Pandey	2003-07-29	Delhi
101	Abhishek Pandey	2003-05-17	Mumbai
102	Aman Pandey	2003-10-23	Delhi
103	Vinay Pandey	2004-02-12	Bangalore
104	Ajay Pandey	2004-06-02	Mumbai

5 rows in set (0.00 sec)

```
mysql> SELECT * FROM Details;
```

EmpID	Salery	Dept
101	75000	BackEnd
102	35000	FrontEnd
104	80000	BackEnd

3 rows in set (0.00 sec)

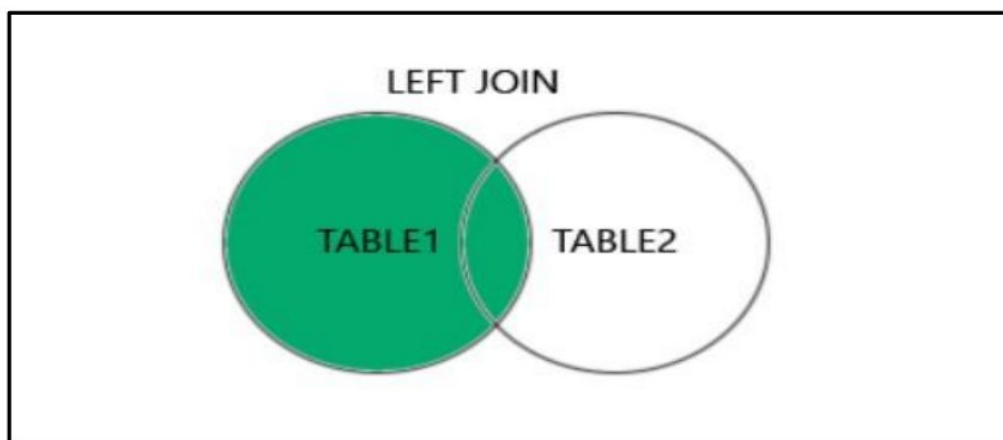
```
mysql> SELECT Y.EmpName , Y.Location , D.Salery , D.Dept FROM (Yash AS Y)  
-> INNER JOIN (Details AS D) ON Y.EmpID = D.EmpID ;
```

EmpName	Location	Salery	Dept
Abhishek Pandey	Mumbai	75000	BackEnd
Aman Pandey	Delhi	35000	FrontEnd
Ajay Pandey	Mumbai	80000	BackEnd

3 rows in set (0.00 sec)

2. Left Join :- The LEFT JOIN keyword selects All records Of Left Table that matching(if any) record in Right table..

SYNTAX ::: SELECT columns FROM table1 LEFT JOIN
table2 ON table1.column = table2.column;



```
mysql> SELECT * FROM Yash;
```

EmpID	EmpName	DOB	Location
100	Yash Pandey	2003-07-29	Delhi
101	Abhishek Pandey	2003-05-17	Mumbai
102	Aman Pandey	2003-10-23	Delhi
103	Vinay Pandey	2004-02-12	Bangalore
104	Ajay Pandey	2004-06-02	Mumbai

5 rows in set (0.00 sec)

```
mysql> SELECT * FROM Details;
```

EmpID	Salery	Dept
101	75000	BackEnd
102	35000	FrontEnd
104	80000	BackEnd

3 rows in set (0.00 sec)

```
mysql> SELECT Y.EmpName , Y.Location , D.Salery , D.Dept FROM  
-> (Yash AS Y) LEFT JOIN (Details AS D) ON Y.EmpID = D.EmpID ;
```

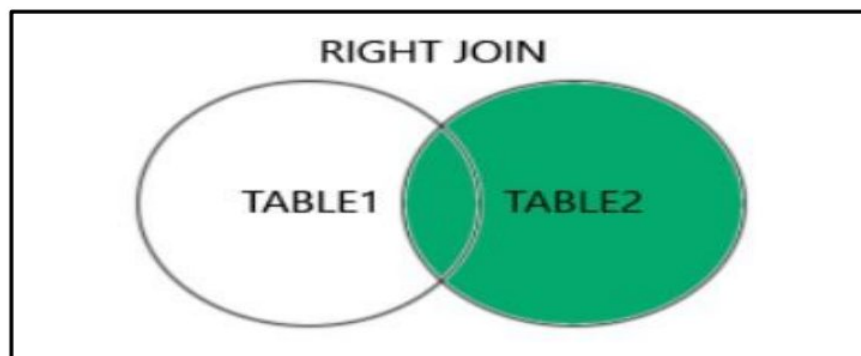
EmpName	Location	Salery	Dept
Yash Pandey	Delhi	NULL	NULL
Abhishek Pandey	Mumbai	75000	BackEnd
Aman Pandey	Delhi	35000	FrontEnd
Vinay Pandey	Bangalore	NULL	NULL
Ajay Pandey	Mumbai	80000	BackEnd

5 rows in set (0.00 sec)

3. Right Join :- The Right JOIN keyword selects All records Of Right Table that matching(if any) record in Left table..

SYNTAX ::: SELECT columns FROM table1 RIGHT JOIN

table2 ON table1.column = table2.column;



```
mysql> SELECT * FROM Yash;
```

EmpID	EmpName	DOB	Location
100	Yash Pandey	2003-07-29	Delhi
101	Abhishek Pandey	2003-05-17	Mumbai
102	Aman Pandey	2003-10-23	Delhi
103	Vinay Pandey	2004-02-12	Bangalore
104	Ajay Pandey	2004-06-02	Mumbai

```
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Details;
```

EmpID	Salery	Dept
101	75000	BackEnd
102	35000	FrontEnd
104	80000	BackEnd

```
3 rows in set (0.00 sec)
```

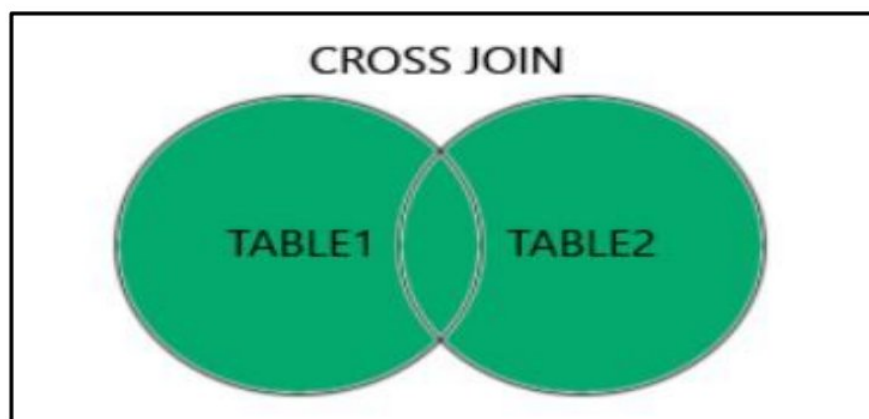
```
mysql> SELECT Y.EmpName , Y.Location , D.Salery , D.Dept FROM  
-> (Yash AS Y) RIGHT JOIN (Details AS D) ON Y.EmpID = D.EmpID ;
```

EmpName	Location	Salery	Dept
Abhishek Pandey	Mumbai	75000	BackEnd
Aman Pandey	Delhi	35000	FrontEnd
Ajay Pandey	Mumbai	80000	BackEnd

```
3 rows in set (0.00 sec)
```

4. Cross Join :- The CROSS JOIN keyword returns all records from both tables (table1 and table2).

SYNTAX ::: SELECT columns FROM table1 CROSS JOIN table2 WHERE table1.column = table2.column;




```
mysql> SELECT * FROM Yash;
```

EmpID	EmpName	DOB	Location
100	Yash Pandey	2003-07-29	Delhi
101	Abhishek Pandey	2003-05-17	Mumbai
102	Aman Pandey	2003-10-23	Delhi
103	Vinay Pandey	2004-02-12	Bangalore
104	Ajay Pandey	2004-06-02	Mumbai

5 rows in set (0.00 sec)

```
mysql> SELECT * FROM Details;
```

EmpID	Salery	Dept
101	75000	BackEnd
102	35000	FrontEnd
104	80000	BackEnd

3 rows in set (0.00 sec)

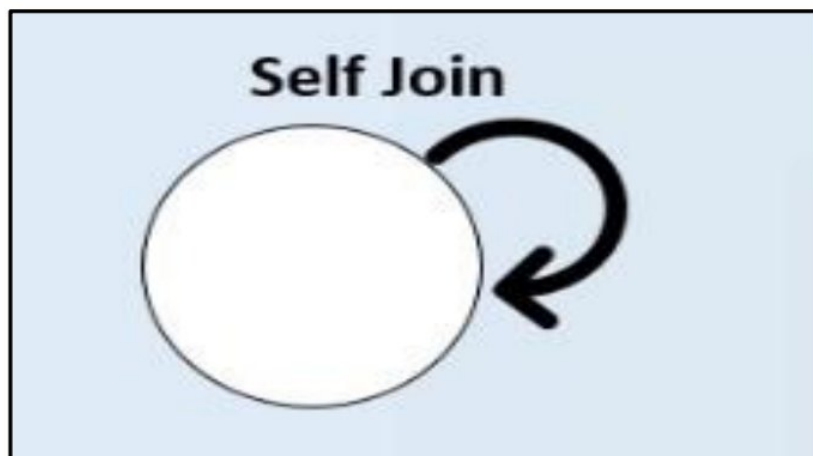
```
mysql> SELECT * FROM Yash CROSS JOIN Details WHERE Yash.EmpID=Details.EmpID;
```

EmpID	EmpName	DOB	Location	EmpID	Salery	Dept
101	Abhishek Pandey	2003-05-17	Mumbai	101	75000	BackEnd
102	Aman Pandey	2003-10-23	Delhi	102	35000	FrontEnd
104	Ajay Pandey	2004-06-02	Mumbai	104	80000	BackEnd

3 rows in set (0.00 sec)

5. Self Join :- The SELF JOIN is a regular join, but the table is joined with itself.

SYNTAX ::: SELECT columns FROM table T1, table T2
WHERE condition;



```
mysql> SELECT * FROM Yash;
```

EmpID	EmpName	DOB	Location
100	Yash Pandey	2003-07-29	Delhi
101	Abhishek Pandey	2003-05-17	Mumbai
102	Aman Pandey	2003-10-23	Delhi
103	Vinay Pandey	2004-02-12	Bangalore
104	Ajay Pandey	2004-06-02	Mumbai

5 rows in set (0.00 sec)

```
mysql> SELECT * FROM Details;
```

EmpID	Salary	Dept
101	75000	BackEnd
102	35000	FrontEnd
104	80000	BackEnd

3 rows in set (0.00 sec)

```
mysql> SELECT Y1.EmpName Emp1 , Y2.EmpName Emp2 , Y2.Location FROM (Yash AS Y1) , (Yash AS Y2)
-> WHERE Y1.EmpID <> Y2.EmpID AND Y1.Location = Y2.Location;
```

Emp1	Emp2	Location
Aman Pandey	Yash Pandey	Delhi
Ajay Pandey	Abhishek Pandey	Mumbai
Yash Pandey	Aman Pandey	Delhi
Abhishek Pandey	Ajay Pandey	Mumbai

4 rows in set (0.00 sec)

LeetCode Questions (Easy).

1. Project Employees - I (Easy #1075)

1075. Project Employees I Solved

Easy Topics Companies

SQL Schema > Pandas Schema >

Table: Project

Column Name	Type
project_id	int
employee_id	int

Code

```
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT P.project_id , ROUND(AVG(experience_years),2) AS
3 average_years FROM (Project AS P) LEFT JOIN (Employee AS E)
4 ON P.employee_id=E.employee_id GROUP BY P.project_id;
```

Saved to local Ln 3, Col 14

Testcase Test Result

2. Product Sales Analysis - I (Easy #1068)

Problem List < > 🔍 Run Submit ⌚ 📄 Premium

Description Editorial Solutions Submissions

1068. Product Sales Analysis I

Solved ✓

Easy Topics Companies

SQL Schema > Pandas Schema >

Table: Sales

Column Name	Type
sale_id	int
product_id	int

Code

```
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT P.product_name, S.year, S.price FROM
3 (Sales AS S) LEFT JOIN (Product AS P) ON
4 S.product_id=P.product_id;
```

Restored from local Upgrade to Cloud Saving Ln 4, Col 1

Testcase Test Result

3. Sales Analysis - III (Easy #1084)

Problem List < > 🔍 Run Submit ⌚ 📄 Premium

Description Editorial Solutions Submissions

1084. Sales Analysis III

Solved ✓

Easy Topics Companies

SQL Schema > Pandas Schema >

Table: Product

Column Name	Type
product_id	int
product_name	varchar
unit_price	int

Code

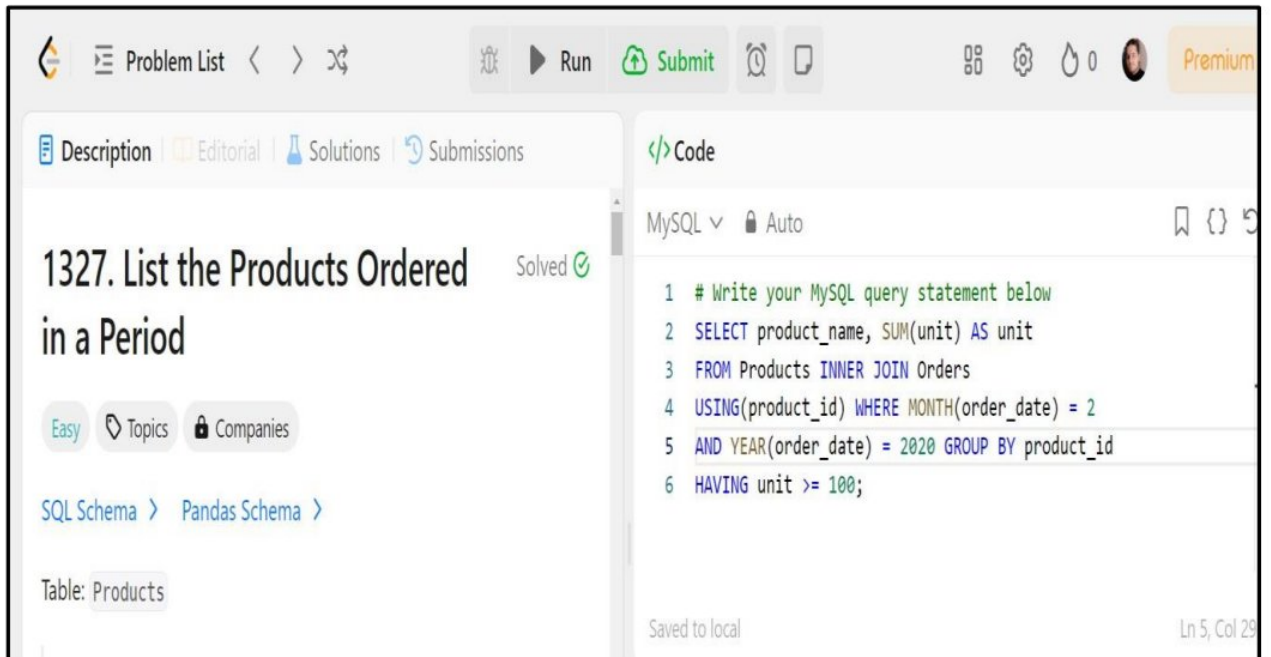
```
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT P.product_id, P.product_name FROM (Product AS P)
3 LEFT JOIN (Sales AS S) ON P.product_id=S.product_id
4 GROUP BY S.product_id HAVING MIN(S.sale_date) >= '2019-01-01'
5 AND MAX(S.sale_date) <= '2019-03-31';
```

Saved to local Ln 5, Col 5

Testcase Test Result

Accepted Runtime: 334 ms

4. List Of Products Problem (Easy #1327)



1327. List the Products Ordered in a Period Solved

Easy Topics Companies

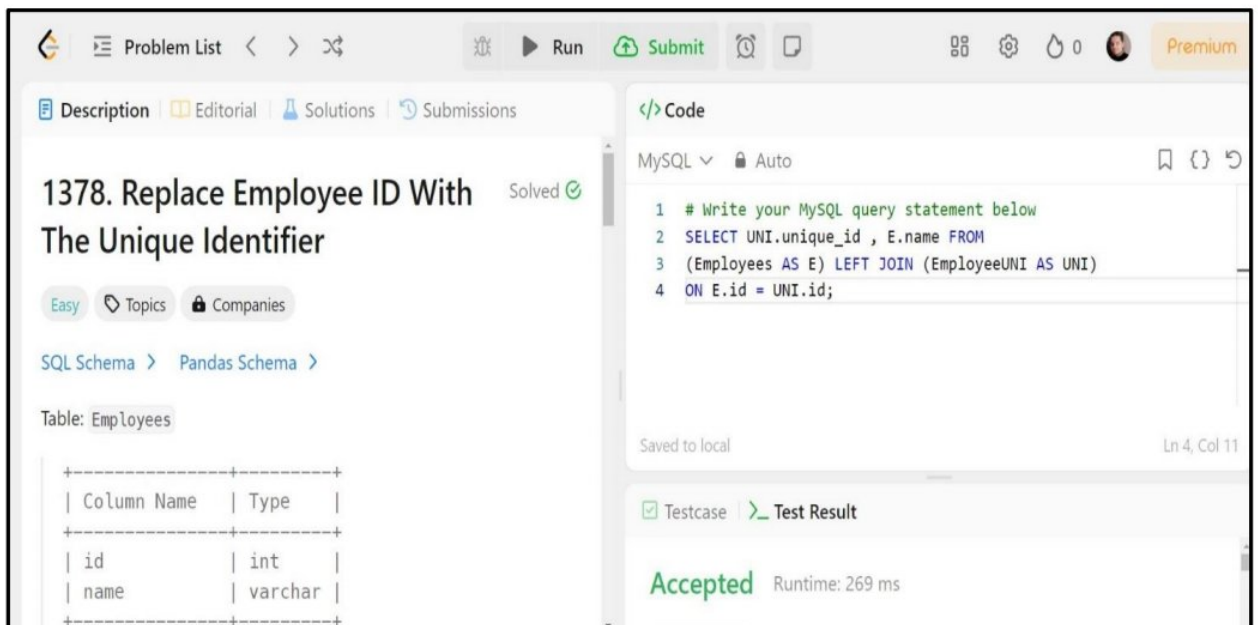
SQL Schema Pandas Schema

Table: Products

```
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT product_name, SUM(unit) AS unit
3 FROM Products INNER JOIN Orders
4 USING(product_id) WHERE MONTH(order_date) = 2
5 AND YEAR(order_date) = 2020 GROUP BY product_id
6 HAVING unit >= 100;
```

Saved to local Ln 5, Col 29

5. Replace EmpID Problem (Easy #1378)



1378. Replace Employee ID With The Unique Identifier Solved

Easy Topics Companies

SQL Schema Pandas Schema

Table: Employees

Column Name	Type
id	int
name	varchar

```
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT UNI.unique_id, E.name FROM
3 (Employees AS E) LEFT JOIN (EmployeeUNI AS UNI)
4 ON E.id = UNI.id;
```

Saved to local Ln 4, Col 11

Testcase Test Result

Accepted Runtime: 269 ms

LeetCode Questions (Medium).

1. Second Highest Salary (Medium #176)

176. Second Highest Salary Solved ✓

Medium Topics Companies

SQL Schema > Pandas Schema >

Table: Employee

Column Name	Type
id	int
salary	int

Code Editor (MySQL):

```
1 # Write your MySQL query statement below
2 SELECT MAX(salary) AS SecondHighestSalary FROM Employee
3 WHERE salary < (SELECT MAX(salary) FROM Employee );
```

Restored from local Upgrade to Cloud Saving Ln 3, Col 7

Testcase Test Result

2. Nth Highest Salary (Medium #177)

177. Nth Highest Salary Solved ✓

Medium Topics Companies

SQL Schema > Pandas Schema >

Table: Employee

Column Name	Type
id	int
salary	int

Code Editor (MySQL):

```
1 CREATE FUNCTION getNthHighestSalary(N INT) RETURNS INT
2 BEGIN
3 SET N = N-1;
4 RETURN (
5     SELECT DISTINCT(salary) FROM Employee
6     ORDER BY salary DESC
7     LIMIT 1 OFFSET N
8 );
9
10 END
```

Saved to local Ln 5, Col 35

Testcase Test Result

3. Department Highest Salary (Medium #184)

Problem List < > Run Submit 0 Premium

Description Editorial Solutions Submissions

184. Department Highest Salary Solved

Medium Topics Companies

SQL Schema > Pandas Schema >

Table: Employee

Column Name	Type
id	int
name	varchar

Code

```
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT D.name AS Department, E.name AS Employee , E.salary
3 FROM (Employee AS E) LEFT JOIN (Department AS D)
4 ON E.departmentId=D.id WHERE (D.Id, E.Salary) IN
5 (SELECT DepartmentId, MAX(Salary) FROM Employee
6 GROUP BY DepartmentId)
```

Saved to local Ln 6, Col 1

☒ Testcase > Test Result

4. Consecutive Numbers (Medium #180)

Problem List < > Run Submit 0 Premium

Description Editorial Solutions Submissions

180. Consecutive Numbers Solved

Medium Topics Companies

SQL Schema > Pandas Schema >

Table: Logs

Column Name	Type
id	int
num	varchar

Code

```
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT DISTINCT l1.num AS ConsecutiveNums
3 FROM logs l1,logs l2,logs l3
4 WHERE l1.id = l2.id+1 AND l2.Id=l3.Id+1
5 AND l1.Num=l2.Num AND l2.Num=l3.Num;
```

Saved to local Ln 3, Col 1

☒ Testcase > Test Result

Accepted Runtime: 220 ms

5. Manager Problem (Medium #570)

Description | Editorial | Solutions | Submissions

570. Managers with at Least 5 Direct Reports

Medium Topics Companies Hint

SQL Schema > Pandas Schema >

Table: Employee

Column Name	Type
id	int
name	varchar

Solved

</> Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 SELECT name FROM Employee WHERE
3 id IN (SELECT managerId FROM Employee
4 GROUP BY managerId HAVING COUNT(managerId)>=5);
```

Saved to local Ln 4, Col 1

☒ Testcase Test Result

Accepted Runtime: 114 ms

6. All Products Problem (Medium #1045)

Description | Editorial | Solutions | Submissions

1045. Customers Who Bought All Products

Medium Topics Companies

SQL Schema > Pandas Schema >

Table: Customer

Column Name	Type
customer_id	int
product_key	int

Solved

</> Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 SELECT customer_id FROM Customer
3 GROUP BY customer_id HAVING
4 COUNT(DISTINCT product_key) =
5 (SELECT COUNT(product_key) FROM Product);
```

Saved to local Ln 5, Col 1

☒ Testcase Test Result

Accepted Runtime: 187 ms

7. Friend Request Problem (Medium #602)

Description Editorial Solutions Submissions

602. Friend Requests II: Who Has the Most Friends

Solved ✓

Medium Topics Companies Hint

SQL Schema > Pandas Schema >

Table: RequestAccepted

Column Name	Type
requester_id	int
accepter_id	int

</> Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 with base as(select requester_id id from RequestAccepted
3 union all
4 select accepter_id id from RequestAccepted)
5 select id, count(*) num from base group by id
6 order by num desc limit 1
```

Saved to local Ln 6, Col 1

Testcase Test Result

Accepted Runtime: 386 ms

8. Tree Node Problem (Medium #608)

Description Editorial Solutions Submissions

608. Tree Node

Solved ✓

Medium Topics Companies Hint

SQL Schema > Pandas Schema >

Table: Tree

Column Name	Type
id	int
p_id	int

id is the column with unique values for this table.

</> Code

MySQL Auto

```
2 SELECT id,
3 CASE
4     WHEN p_id IS NULL THEN 'Root'
5     WHEN id IN (SELECT p_id FROM Tree) THEN 'Inner'
6     ELSE 'Leaf'
7 END AS type
8 FROM Tree
9
```

Saved to local Ln 8, Col 1

Testcase Test Result

Accepted Runtime: 336 ms

9. Product Sales Analysis-III (Medium #1070)

Description | Editorial | Solutions | Submissions

1070. Product Sales Analysis III Solved

Medium Topics Companies

SQL Schema > Pandas Schema >

Table: Sales

Column Name	Type
sale_id	int
product_id	int
year	int

</> Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 SELECT product_id, year AS first_year, quantity, price
3 FROM sales WHERE
4 (product_id, year) IN (SELECT product_id, MIN(year)
5 FROM sales GROUP BY product_id);
```

Saved to local Ln 5, Col 12

☒ Testcase > Test Result

Accepted Runtime: 230 ms

10. Food Delivery-II (Medium #1174)

Description | Editorial | Solutions | Submissions

1174. Immediate Food Delivery II Solved

Medium Topics Companies

SQL Schema > Pandas Schema >

Table: Delivery

Column Name	Type
delivery_id	int
customer_id	int

</> Code

MySQL Auto

```
2 select
3 ROUND(AVG(order_date = customer_pref_delivery_date) * 100, 2)
4 as immediate_percentage
5 from delivery where (customer_id, order_date) IN
6 (select customer_id, min(order_date) as first_order
7 from delivery
8 group by customer_id)
```

Saved to local Ln 3, Col 21

☒ Testcase > Test Result

Accepted Runtime: 184 ms

11. Count Salary Problem (Medium #1907)

Description | Editorial | Solutions | Submissions

1907. Count Salary Categories

Medium | Topics | Companies

SQL Schema > Pandas Schema >

Table: Accounts

Column Name	Type
account_id	int
income	int

Solved

</> Code

MySQL Auto

```
1 SELECT "Low Salary" as category, sum(if(income<20000,1,0))
2 AS accounts_count FROM Accounts
3 union
4 SELECT "Average Salary" as category, sum(if(income>=20000
5 and income<=50000,1,0)) AS accounts_count FROM Accounts
6 union
7 SELECT "High Salary" as category, sum(if(income>50000,1,0))
8 AS accounts count FROM Accounts
```

Saved to local Ln 5, Col 25

☒ Testcase | Test Result

Accepted Runtime: 233 ms

12. Monthly Transactions - I (Medium #1193)

Description | Editorial | Solutions | Submissions

1193. Monthly Transactions I

Medium | Topics | Companies

SQL Schema > Pandas Schema >

Table: Transactions

Column Name	Type
id	int
country	varchar
state	enum

Solved

</> Code

MySQL Auto

```
1 SELECT DATE_FORMAT(trans_date, '%Y-%m') AS month,
2 country, COUNT(id) AS trans_count,
3 COUNT(IF(state = 'approved', 1, NULL)) AS approved_count,
4 SUM(amount) AS trans_total_amount,
5 SUM(IF(state='approved', amount, 0)) AS approved_total_amount
6 FROM Transactions
7 GROUP BY 1, 2 ;
```

Saved to local Ln 7, Col 16

☒ Testcase | Test Result

Accepted Runtime: 205 ms

13. Bus Capacity Problem (Medium #1204)

Description | Editorial | Solutions | Submissions

1204. Last Person to Fit in the Bus

Medium | Topics | Companies

SQL Schema > Pandas Schema >

Table: Queue

Column Name	Type
person_id	int
person_name	varchar

Solved

</> Code

MySQL Auto

1 SELECT q1.person_name
2 FROM Queue q1 JOIN Queue q2 ON q1.turn >= q2.turn
3 GROUP BY q1.turn HAVING
4 SUM(q2.weight) <= 1000 ORDER BY
5 SUM(q2.weight) DESC LIMIT 1;

Saved to local

Ln 3, Col 25

Testcase

Test Result

Accepted Runtime: 162 ms

14. Restaurant Growth Problem (Medium #1321)

Description | Editorial | Solutions | Submissions

1321. Restaurant Growth

Medium | Topics | Companies

SQL Schema > Pandas Schema >

Table: Customer

Column Name	Type
customer_id	int
name	varchar
visited_on	date

Solved

</> Code

MySQL Auto

1 SELECT visited_on, amount, ROUND(amount/7, 2) average_amount
2 FROM (SELECT DISTINCT visited_on, SUM(amount)
3 OVER(ORDER BY visited_on RANGE BETWEEN INTERVAL 6 DAY
4 PRECEDING AND CURRENT ROW) amount, MIN(visited_on)
5 OVER() 1st_date FROM Customer) AS t
6 WHERE visited_on >= 1st_date+6;

Saved to local

Ln 5, Col 17

Testcase

Test Result

Accepted Runtime: 193 ms

15. Capital Loss/Gain (Medium #1393)

Description | Editorial | Solutions | Submissions

1393. Capital Gain/Loss

Medium | Topics | Companies

SQL Schema > Pandas Schema >

Table: Stocks

Column Name	Type
stock_name	varchar
operation	enum
operation_day	int
price	int

Solved

Code

MySQL Auto

```
1 SELECT stock_name, SUM(  
2     CASE  
3         WHEN operation = 'Buy' THEN -price  
4         ELSE price  
5     END  
6 ) AS capital_gain_loss FROM Stocks  
7 GROUP BY stock_name
```

Saved to local

Ln 6, Col 24

Testcase

Test Result

Accepted Runtime: 152 ms

LeetCode Profile Page



Yash Pandey
pandeyyash041

Rank 1,886,811

college Sophomore At MAIT CSE ,
Preparing for Top MNC , Learning Java with
DSA And Curious About Web Development

Edit Profile

Solved Problems

40
Solved

Easy	25 / 771	Beats 59.0%
Medium	15 / 1599	Beats 50.9%
Hard	0 / 675	Not enough data

70 submissions in the last year ⓘ

Languages

MySQL

38 problems solved

Java

2 problems solved

Skills

Advanced

Not enough data

Intermediate

Database x38

Bit Manipulation x1

Fundamental

Array x2

Sorting x1

Two Pointers x1

Lab Exercise-5. (06/03/2024)

(Problem Statement)

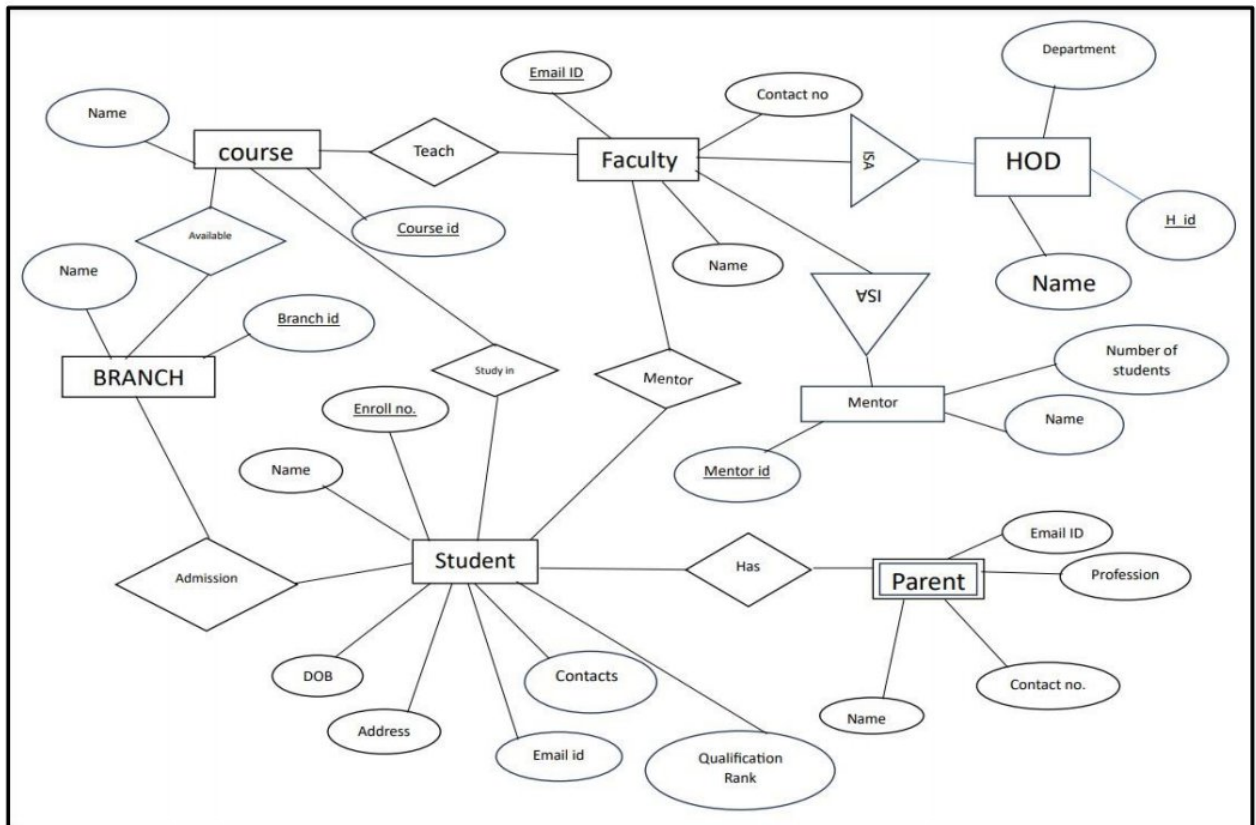
Q :- An Institute offers multiple programmes Students admitted in the programme can enroll in many courses. Institute has a Director and Hod of each programme. Faculties recruited for a programme can teach only a single course in multiple programmes. For each course there is a faculty coordinator responsible for coordinating all the faculties teaching the same course . A faculty can be mentor of 20 students. however, a student can have only one mentor. The mentor takes counsellor meeting twice a month. Data to be recorded in the database include :-

Student - Enrollment no. , name, data of birth. address. contact no. email - id. qualifying exam rank, programme Faculty — Name, contact no, email id. programme, Course Parents data - Parents name, contact no. email id, Profession .

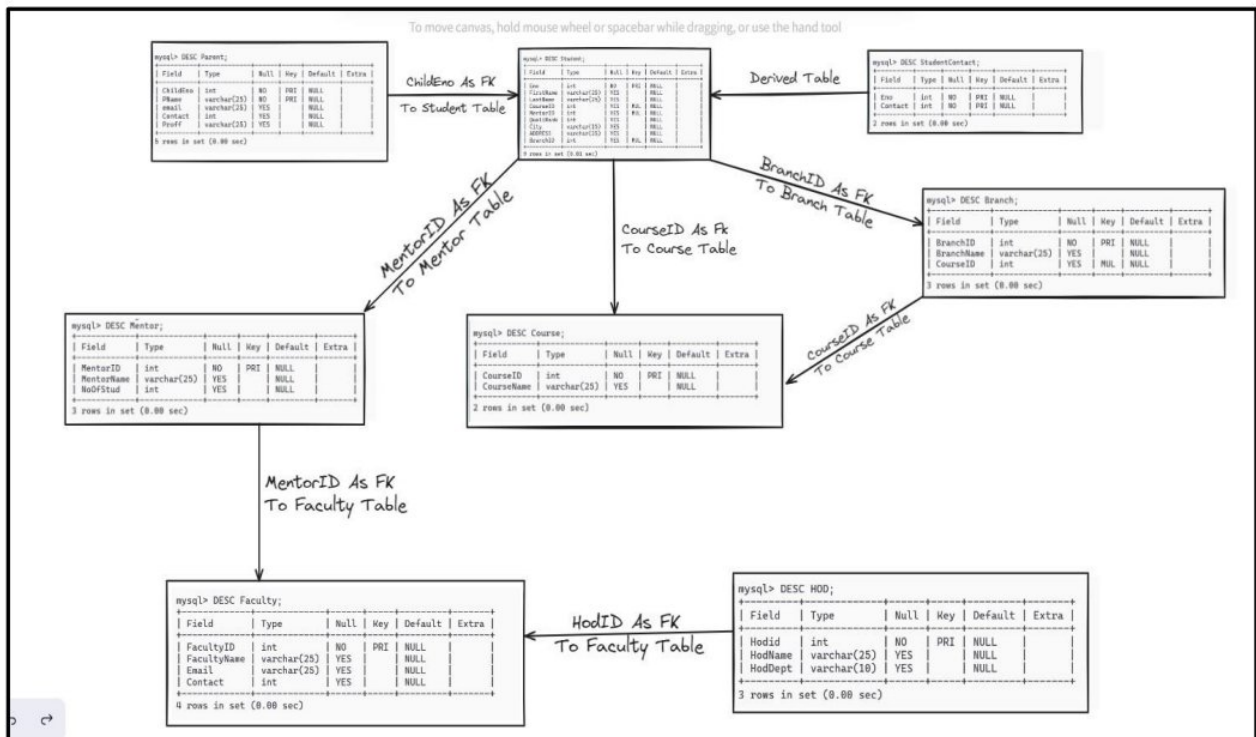
Tasks To Be Performed Are :-

- Draw The ER Diagram Of Given Statement .
- Build Logical And Physical Design .
- Create The Database For Above Problem .
- Apply The Required Integrity Constraints .
- Create The Instance Of The Database .
- Check All Integrity Constraints Working Properly

1. ER Model For Above Problem



2. Logical And Physical Design



- Structure Of Student , Parent , Faculty & StudentContact Tables**

```
mysql> DESC Student;
```

Field	Type	Null	Key	Default	Extra
Eno	int	NO	PRI	NULL	
FirstName	varchar(25)	YES		NULL	
LastName	varchar(25)	YES		NULL	
CourseID	int	YES	MUL	NULL	
MentorID	int	YES	MUL	NULL	
QualiRank	int	YES		NULL	
City	varchar(15)	YES		NULL	
ADDRESS	varchar(25)	YES		NULL	
BranchID	int	YES	MUL	NULL	

9 rows in set (0.01 sec)

```
mysql> DESC Faculty;
```

Field	Type	Null	Key	Default	Extra
FacultyID	int	NO	PRI	NULL	
FacultyName	varchar(25)	YES		NULL	
Email	varchar(25)	YES		NULL	
Contact	int	YES		NULL	

4 rows in set (0.00 sec)

```
mysql> DESC Parent;
```

Field	Type	Null	Key	Default	Extra
ChildEno	int	NO	PRI	NULL	
PName	varchar(25)	NO	PRI	NULL	
email	varchar(25)	YES		NULL	
Contact	int	YES		NULL	
Proff	varchar(25)	YES		NULL	

5 rows in set (0.00 sec)

```
mysql> DESC StudentContact;
```

Field	Type	Null	Key	Default	Extra
Eno	int	NO	PRI	NULL	
Contact	int	NO	PRI	NULL	

2 rows in set (0.00 sec)

- Structure Of Course , Branch , Mentor & HOD Tables**

```
mysql> DESC Course;
```

Field	Type	Null	Key	Default	Extra
CourseID	int	NO	PRI	NULL	
CourseName	varchar(25)	YES		NULL	

2 rows in set (0.00 sec)

```
mysql> DESC Mentor;
```

Field	Type	Null	Key	Default	Extra
MentorID	int	NO	PRI	NULL	
MentorName	varchar(25)	YES		NULL	
NoOfStud	int	YES		NULL	

3 rows in set (0.00 sec)

```
mysql> DESC Branch;
```

Field	Type	Null	Key	Default	Extra
BranchID	int	NO	PRI	NULL	
BranchName	varchar(25)	YES		NULL	
CourseID	int	YES	MUL	NULL	

3 rows in set (0.00 sec)

```
mysql> DESC HOD;
```

Field	Type	Null	Key	Default	Extra
Hodid	int	NO	PRI	NULL	
HodName	varchar(25)	YES		NULL	
HodDept	varchar(10)	YES		NULL	

3 rows in set (0.00 sec)

3. Database Design

```
mysql> CREATE TABLE Student( Eno INT(10) , FirstName VARCHAR(25) , LastName VARCHAR(25) , CourseID INT(10) ,  
-> MentorID INT(10) , QualiRank INT(10) , City VARCHAR(15) , ADDRESS VARCHAR(25) , BranchID INT(10) );  
Query OK, 0 rows affected, 5 warnings (0.07 sec)  
  
mysql> CREATE TABLE StudentContact( Eno INT(10) , Contact INT(10) );  
Query OK, 0 rows affected, 2 warnings (0.05 sec)  
  
mysql> CREATE TABLE Parent( ChildEno INT(10) , PName VARCHAR(25) , email VARCHAR(25) , Contact INT(10) , Proff VARCHAR(25) );  
Query OK, 0 rows affected, 2 warnings (0.04 sec)  
  
mysql> CREATE TABLE Branch( BranchID INT(10) , BranchName VARCHAR(25) , CourseID INT(10) );  
Query OK, 0 rows affected, 2 warnings (0.07 sec)  
  
mysql> CREATE TABLE Course( CourseID INT(10) , CourseName VARCHAR(25) );  
Query OK, 0 rows affected, 1 warning (0.07 sec)  
  
mysql> CREATE TABLE Faculty( FacultyID INT(10) , FacultyName VARCHAR(25) , Email VARCHAR(25) , Contact INT(10) );  
Query OK, 0 rows affected, 2 warnings (0.04 sec)  
  
mysql> CREATE TABLE MENTOR( MentorID INT(10) , MentorName VARCHAR(25) , NoOfStud INT(5) );  
Query OK, 0 rows affected, 2 warnings (0.09 sec)  
  
mysql> CREATE TABLE Hod(Hodid INT(10) , HodName VARCHAR(25) , HodDept VARCHAR(10) );  
Query OK, 0 rows affected, 1 warning (0.04 sec)
```

4. Integrity Constraints

```
mysql> ALTER TABLE STUDENT ADD PRIMARY KEY(Eno);  
Query OK, 0 rows affected (0.12 sec)  
Records: 0 Duplicates: 0 Warnings: 0  
  
mysql> ALTER TABLE StudentContact ADD PRIMARY KEY(Eno,Contact);  
Query OK, 0 rows affected (0.09 sec)  
Records: 0 Duplicates: 0 Warnings: 0  
  
mysql> ALTER TABLE PARENT ADD PRIMARY KEY(ChildEno,Pname);  
Query OK, 0 rows affected (0.08 sec)  
Records: 0 Duplicates: 0 Warnings: 0  
  
mysql> ALTER TABLE BRANCH ADD PRIMARY KEY(BranchID);  
Query OK, 0 rows affected (0.08 sec)  
Records: 0 Duplicates: 0 Warnings: 0  
  
mysql> ALTER TABLE Course ADD PRIMARY KEY(CourseID);  
Query OK, 0 rows affected (0.11 sec)  
Records: 0 Duplicates: 0 Warnings: 0  
  
mysql> ALTER TABLE Faculty ADD PRIMARY KEY(FacultyID);  
Query OK, 0 rows affected (0.07 sec)  
Records: 0 Duplicates: 0 Warnings: 0  
  
mysql> ALTER TABLE Mentor ADD PRIMARY KEY(MentorID);  
Query OK, 0 rows affected (0.11 sec)  
Records: 0 Duplicates: 0 Warnings: 0  
  
mysql> ALTER TABLE HOD ADD PRIMARY KEY(HodID);  
Query OK, 0 rows affected (0.07 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```

mysql> ALTER TABLE Student ADD FOREIGN KEY(CourseID) references Course(CourseID) ;
Query OK, 0 rows affected (0.14 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE Student ADD FOREIGN KEY(MentorID) references Mentor(MentorID) ;
Query OK, 0 rows affected (0.18 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE Student ADD FOREIGN KEY(BranchID) references Branch(BranchID) ;
Query OK, 0 rows affected (0.20 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE Parent ADD FOREIGN KEY(ChildEno) references STUDENT(Eno) ;
Query OK, 0 rows affected (0.12 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE Branch ADD FOREIGN KEY(CourseID) references Course(CourseID) ;
Query OK, 0 rows affected (0.16 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE Mentor ADD FOREIGN KEY(MentorID) references Faculty(FacultyID) ;
Query OK, 0 rows affected (0.16 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE Hod ADD FOREIGN KEY(HodID) references Faculty(FacultyID);
Query OK, 0 rows affected (0.10 sec)
Records: 0 Duplicates: 0 Warnings: 0

```

5. Instance Creation In The 8 Tables

```

mysql> INSERT INTO Course VALUES(1110,"B.Tech");
Query OK, 1 row affected (0.02 sec)

mysql> INSERT INTO Course VALUES(1111,"BCA");
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Branch VALUES(11101,"CSE",1110);
Query OK, 1 row affected (0.04 sec)

mysql> INSERT INTO Branch VALUES(11102,"IT",1110);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Branch VALUES(11103,"ECE",1110);
Query OK, 1 row affected (0.04 sec)

mysql> INSERT INTO Branch VALUES(11111,"Computer",1111);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Faculty VALUES(1001,"Aman","Aman@gmail.com",9745681237);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Faculty VALUES(1002,"Rahul","Rahul@gmail.com",8765681237);
Query OK, 1 row affected (0.04 sec)

mysql> INSERT INTO Faculty VALUES(1003,"Ankit","Ankit@gmail.com",9435681237);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Faculty VALUES(1004,"Pankaj","Pankaj@gmail.com",9495681237);
Query OK, 1 row affected (0.04 sec)

mysql> INSERT INTO Mentor VALUES(1001,"Aman",2);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Mentor VALUES(1003,"Ankit",2);
Query OK, 1 row affected (0.03 sec)

mysql> INSERT INTO Mentor VALUES(1004,"Pankaj",1);
Query OK, 1 row affected (0.01 sec)

```

```

mysql> INSERT INTO Student Values(101,"Yash","Pandey",1110,1001,45271,"Delhi","Rohini",11101,'2003-07-29');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Student Values(102,"Abhishek","Pandey",1110,1003,18211,"Delhi","NSP",11101,'2003-02-12');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Student Values(103,"Yash","Kumar",1110,1003,68287,"Delhi","Uttam Nagar",11101,'2004-03-18');
Query OK, 1 row affected (0.04 sec)

mysql> INSERT INTO Student Values(104,"Arya","Gautam",1110,1001,57582,"Delhi","VikasPuri",11102,'2003-06-18');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Student Values(105,"Neeraj","Kumar",1110,1004,27582,"Delhi","Mangolpuri",11103,'2003-09-08');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO StudentContact VALUES(101,9745862415);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO StudentContact VALUES(101,8745862415);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO StudentContact VALUES(102,8765862475);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO StudentContact VALUES(103,9713815247);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO StudentContact VALUES(104,8713824247);
Query OK, 1 row affected (0.04 sec)

mysql> INSERT INTO StudentContact VALUES(104,8513824247);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO StudentContact VALUES(105,9513224247);
Query OK, 1 row affected (0.04 sec)

```

```

mysql> INSERT INTO Parent VALUES(101,"Prabhash Pandey","Prabhash@gmail.com",9724615248,"Govt. Job");
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Parent VALUES(101,"Rakesh Kumar","rakesh@gmail.com",8753615248,"Doctor");
Query OK, 1 row affected (0.01 sec)

mysql> UPDATE Parent SET ChildEno=102 WHERE PName="Rakesh Kumar";
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> INSERT INTO Parent VALUES(103,"Rahul Pandey","Rahul@gmail.com",8464615248,"Govt. Job");
Query OK, 1 row affected (0.04 sec)

mysql> INSERT INTO Parent VALUES(104,"Suresh Kumar","Suresh@gmail.com",8467815248,"Cab Driver");
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Parent VALUES(105,"Aman Pandey","Aman@gmail.com",7548615248,"IT Officer");
Query OK, 1 row affected (0.01 sec)

```


6. Checking All Integrity Constraints Working Properly

- Yes . All Integrity Constraints Are Working Properly .
- The Database Is Been Designed .

```
mysql> SELECT * From STUDENT;
```

Eno	FirstName	LastName	CourseID	MentorID	QualiRank	City	ADDRESS	BranchID	DOB
101	Yash	Pandey	1110	1001	45271	Delhi	Rohini	11101	2003-07-29
102	Abhishek	Pandey	1110	1003	18211	Delhi	NSP	11101	2003-02-12
103	Yash	Kumar	1110	1003	68287	Delhi	Uttam Nagar	11102	2004-03-18
104	Arya	Gautam	1110	1001	57582	Delhi	VikasPuri	11102	2003-06-18
105	Neeraj	Kumar	1110	1004	27582	Delhi	Mangolpuri	11103	2003-09-08

5 rows in set (0.00 sec)

```
mysql> SELECT * FROM StudentContact;
```

Eno	Contact
101	8745862415
101	9745862415
102	8765862475
103	9713815247
104	8513824247
104	8713824247
105	9513224247

7 rows in set (0.00 sec)

```
mysql> SELECT * FROM Parent;
```

ChildEno	PName	email	Contact	Proff
101	Prabhash Pandey	Prabhash@gmail.com	9724615248	Govt. Job
102	Rakesh Kumar	rakesh@gmail.com	8753615248	Doctor
103	Rahul Pandey	Rahul@gmail.com	8464615248	Govt. Job
104	Suresh Kumar	Suresh@gmail.com	8467815248	Cab Driver
105	Aman Pandey	Aman@gmail.com	7548615248	IT Officer

5 rows in set (0.00 sec)

```
mysql> SELECT * FROM Branch;
```

BranchID	BranchName	CourseID
11101	CSE	1110
11102	IT	1110
11103	ECE	1110
11111	Computer	1111

4 rows in set (0.00 sec)

```
mysql> SELECT * FROM Course;
```

CourseID	CourseName
1110	B.Tech
1111	BCA

2 rows in set (0.00 sec)

```
mysql> SELECT * FROM Faculty;
```

FacultyID	FacultyName	Email	Contact
1001	Aman	Aman@gmail.com	9745681237
1002	Rahul	Rahul@gmail.com	8765681237
1003	Ankit	Ankit@gmail.com	9435681237
1004	Pankaj	Pankaj@gmail.com	9495681237

4 rows in set (0.00 sec)


```
mysql> SELECT * FROM Mentor;
```

MentorID	MentorName	NoOfStud
1001	Aman	2
1003	Ankit	2
1004	Pankaj	1

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM HOD;
```

Hodid	HodName	HodDept
1001	Aman	CSE
1002	Rahul	IT
1004	Pankaj	ECE

```
3 rows in set (0.00 sec)
```

Project (Task-1.)

(13/03/2024)

Q :- Check the highest Normalized form of each relation and take appropriate action Required ?

1. Student Table → BCNF Normalized

```
mysql> SELECT * FROM STUDENT;
```

Eno	FirstName	LastName	CourseID	MentorID	QualiRank	City	ADDRESS	BranchID	DOB
101	Yash	Pandey	1110	1001	45271	Delhi	Rohini	11101	2003-07-29
102	Abhishek	Pandey	1110	1003	18211	Delhi	NSP	11101	2003-02-12
103	Yash	Kumar	1110	1003	68287	Delhi	Uttam Nagar	11102	2004-03-18
104	Arya	Gautam	1110	1001	57582	Delhi	VikasPuri	11102	2003-06-18
105	Neeraj	Kumar	1110	1004	27582	Delhi	Mangolpuri	11103	2003-09-08

5 rows in set (0.01 sec)

2. StudentContact Table → BCNF Normalized

```
mysql> SELECT * FROM STUDENTCONTACT;
```

Eno	Contact
101	8745862415
101	9745862415
102	8765862475
103	9713815247
104	8513824247
104	8713824247
105	9513224247

7 rows in set (0.00 sec)

3. Parent Table → BCNF Normalized

```
mysql> SELECT * FROM PARENT;
```

ChildEno	PName	email	Contact	Proff
101	Prabhash Pandey	Prabhash@gmail.com	9724615248	Govt. Job
102	Rakesh Kumar	rakesh@gmail.com	8753615248	Doctor
103	Rahul Pandey	Rahul@gmail.com	8464615248	Govt. Job
104	Suresh Kumar	Suresh@gmail.com	8467815248	Cab Driver
105	Aman Pandey	Aman@gmail.com	7548615248	IT Officer

```
5 rows in set (0.00 sec)
```

4. Course Table → BCNF Normalized

```
mysql> SELECT * FROM COURSE;
```

CourseID	CourseName
1110	B.Tech
1111	BCA

```
2 rows in set (0.01 sec)
```

5. Branch Table → BCNF Normalized

```
mysql> SELECT * FROM Branch;
```

BranchID	BranchName	CourseID
11101	CSE	1110
11102	IT	1110
11103	ECE	1110
11111	Computer	1111

```
4 rows in set (0.00 sec)
```

6. Creation Of Subject Table To Store Subjects

```
mysql> CREATE TABLE Subjects(SCode VARCHAR(10) Primary Key , SName VARCHAR(25) NOT NULL);
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO Subjects VALUES("BS-202","Probability");
Query OK, 1 row affected (0.02 sec)

mysql> INSERT INTO Subjects VALUES("CIC-206","Theory Of Computation");
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Subjects VALUES("CIC-208","Circuit & Systems");
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Subjects VALUES("CIC-210","DBMS");
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Subjects VALUES("CIC-212","Prog. In Java");
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Subjects;
+-----+-----+
| SCode | SName |
+-----+-----+
| BS-202 | Probability |
| CIC-206 | Theory Of Computation |
| CIC-208 | Circuit & Systems |
| CIC-210 | DBMS |
| CIC-212 | Prog. In Java |
+-----+-----+
5 rows in set (0.00 sec)
```

7. Modify Faculty Table To Avoid Redundancy

```
mysql> ALTER TABLE Faculty ADD COLUMN BranchID VARCHAR(5);
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE Faculty ADD COLUMN SCode VARCHAR(10);
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE Faculty ADD FOREIGN KEY(BranchID) references Branch(BranchID);
Query OK, 4 rows affected (0.15 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE Faculty ADD FOREIGN KEY(SCode) references Subjects(SCode);
Query OK, 4 rows affected (0.14 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> DESC Faculty;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| FacultyID | int | NO | PRI | NULL | |
| FacultyName | varchar(25) | YES | | NULL | |
| Email | varchar(25) | YES | | NULL | |
| Contact | text | YES | | NULL | |
| BranchID | int | YES | MUL | NULL | |
| SCode | varchar(10) | YES | MUL | NULL | |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.02 sec)
```



```

mysql> UPDATE Faculty SET BranchID=11101 , SCode="CIC-210" WHERE FacultyID=101;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql> UPDATE Faculty SET BranchID=11101 , SCode="CIC-210" WHERE FacultyID=1001;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE Faculty SET BranchID=11101 , SCode="CIC-212" WHERE FacultyID=1002;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE Faculty SET BranchID=11102 , SCode="CIC-208" WHERE FacultyID=1003;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE Faculty SET BranchID=11102 , SCode="CIC-208" WHERE FacultyID=1004;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE Faculty SET BranchID=11103 , SCode="CIC-206" WHERE FacultyID=1005;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql> SELECT * FROM Faculty;
+-----+-----+-----+-----+-----+-----+
| FacultyID | FacultyName | Email | Contact | BranchID | SCode |
+-----+-----+-----+-----+-----+-----+
| 1001 | Aman | Aman@gmail.com | 9745681237 | 11101 | CIC-210 |
| 1002 | Rahul | Rahul@gmail.com | 8765681237 | 11101 | CIC-212 |
| 1003 | Ankit | Ankit@gmail.com | 9435681237 | 11102 | CIC-208 |
| 1004 | Pankaj | Pankaj@gmail.com | 9495681237 | 11102 | CIC-208 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

```

8. Create Role Table To Define Roles

```

mysql> CREATE TABLE Role(RoleName VARCHAR(5) Primary Key , Description VARCHAR(25) NOT NULL);
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO Role Values("Mtr","Mentor Of Students");
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Role Values("H-CSE","HOD Of CSE Dept.");
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Role Values("H-IT","HOD Of IT Dept.");
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Role Values("H-ECE","HOD Of ECE Dept.");
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Role;
+-----+-----+
| RoleName | Description |
+-----+-----+
| H-CSE | HOD Of CSE Dept. |
| H-ECE | HOD Of ECE Dept. |
| H-IT | HOD Of IT Dept. |
| Mtr | Mentor Of Students |
+-----+-----+
4 rows in set (0.00 sec)

```

9. Create FacultyRole Table To Assign Roles

```
mysql> CREATE TABLE FacultyRole(FacultyID INT , Role VARCHAR(5) NOT NULL);
Query OK, 0 rows affected (0.05 sec)

mysql> ALTER TABLE FacultyRole ADD PRIMARY KEY(FacultyID,Role);
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE FacultyRole ADD FOREIGN KEY(FacultyID) References Faculty(FacultyID);
Query OK, 0 rows affected (0.10 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> INSERT INTO FacultyRole VALUES(1001,"Mtr");
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO FacultyRole VALUES(1001,"H-CSE");
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO FacultyRole VALUES(1002,"H-IT");
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO FacultyRole VALUES(1003,"Mtr");
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO FacultyRole VALUES(1004,"Mtr");
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO FacultyRole VALUES(1004,"H-ECE");
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM FacultyRole;
+-----+-----+
| FacultyID | Role |
+-----+-----+
| 1001      | H-CSE |
| 1001      | Mtr   |
| 1002      | H-IT  |
| 1003      | Mtr   |
| 1004      | H-ECE |
| 1004      | Mtr   |
+-----+-----+
6 rows in set (0.00 sec)
```

- The Database Is Been Designed And Normalized Where Every Table Of Above Table Is BCNF Normalized .

```
mysql> SHOW TABLES;
+-----+
| Tables_in_institutes |
+-----+
| branch                |
| course                |
| faculty               |
| facultyrole           |
| parent               |
| role                  |
| student               |
| studentcontact        |
| subjects              |
+-----+
11 rows in set (0.01 sec)
```

➤ Every Table Is Been Designed And BCNF Normalized .

```
mysql> SELECT * FROM STUDENT;
```

Eno	FirstName	LastName	CourseID	MentorID	QualiRank	City	ADDRESS	BranchID	DOB
101	Yash	Pandey	1110	1001	45271	Delhi	Rohini	11101	2003-07-29
102	Abhishek	Pandey	1110	1003	18211	Delhi	NSP	11101	2003-02-12
103	Yash	Kumar	1110	1003	68287	Delhi	Uttam Nagar	11102	2004-03-18
104	Arya	Gautam	1110	1001	57582	Delhi	VikasPuri	11102	2003-06-18
105	Neeraj	Kumar	1110	1004	27582	Delhi	Mangolpuri	11103	2003-09-08

```
5 rows in set (0.01 sec)
```

```
mysql> SELECT * FROM PARENT;
```

ChildEno	PName	email	Contact	Proff
101	Prabhash Pandey	Prabhash@gmail.com	9724615248	Govt. Job
102	Rakesh Kumar	rakesh@gmail.com	8753615248	Doctor
103	Rahul Pandey	Rahul@gmail.com	8464615248	Govt. Job
104	Suresh Kumar	Suresh@gmail.com	8467815248	Cab Driver
105	Aman Pandey	Aman@gmail.com	7548615248	IT Officer

```
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM StudentContact;
```

Eno	Contact
101	8745862415
101	9745862415
102	8765862475
103	9713815247
104	8513824247
104	8713824247
105	9513224247

```
7 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Role;
```

RoleName	Description
H-CSE	HOD Of CSE Dept.
H-ECE	HOD Of ECE Dept.
H-IT	HOD Of IT Dept.
Mtr	Mentor Of Students

```
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM FacultyRole;
```

FacultyID	Role
1001	H-CSE
1001	Mtr
1002	H-IT
1003	Mtr
1004	H-ECE
1004	Mtr

```
6 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM FACULTY;
```

FacultyID	FacultyName	Email	Contact	BranchID	SCode
1001	Aman	Aman@gmail.com	9745681237	11101	CIC-210
1002	Rahul	Rahul@gmail.com	8765681237	11101	CIC-212
1003	Ankit	Ankit@gmail.com	9435681237	11102	CIC-208
1004	Pankaj	Pankaj@gmail.com	9495681237	11102	CIC-208

```
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM COURSE;
```

CourseID	CourseName
1110	B.Tech
1111	BCA

```
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM BRANCH;
```

BranchID	BranchName	CourseID
11101	CSE	1110
11102	IT	1110
11103	ECE	1110
11111	Computer	1111

```
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM SUBJECTS;
```

SCode	SName
BS-202	Probability
CIC-206	Theory Of Computation
CIC-208	Circuit & Systems
CIC-210	DBMS
CIC-212	Prog. In Java

```
5 rows in set (0.00 sec)
```


Project(Task-2.)

(13/03/2024)

Q :- Write SQL , Relational Query for above Database

1. List enrolment no. and student names programme-wise

① (i) σ ENO , FirstName, LastName , Branch Name (Student $\bowtie$ Branch);

```
mysql> SELECT S.ENO , S.FirstName , S.LastName , B.BranchName FROM  
-> (Student S) LEFT JOIN (Branch B) On S.BranchID = B.BranchID;
```

ENO	FirstName	LastName	BranchName
101	Yash	Pandey	CSE
102	Abhishek	Pandey	CSE
103	Yash	Kumar	IT
104	Arya	Gautam	IT
105	Neeraj	Kumar	ECE

5 rows in set (0.03 sec)

2. List Programme-Wise Student Names .

② Branch Name , FirstName, LastName γ (Student $\bowtie$ Branch)

```
mysql> SELECT B.BranchName , S.FirstName , S.LastName FROM (Student S)  
-> LEFT JOIN (Branch B) On S.BranchID = B.BranchID;
```

BranchName	FirstName	LastName
CSE	Yash	Pandey
CSE	Abhishek	Pandey
IT	Yash	Kumar
IT	Arya	Gautam
ECE	Neeraj	Kumar

5 rows in set (0.00 sec)

3. Display Total No. Of Students Taught By Each Faculty .

③ $\rho(T_1, \text{Student})$, $\rho(T_2, \text{Faculty})$
 $\rho(\text{FacultyName } F, \text{count(ENO)}) (T_1 \bowtie T_2)$
 $T_1.\text{MentorID} = T_2.\text{FacultyID}$

```
mysql> SELECT F.FacultyName , COUNT(St.ENO) FROM (Student St) LEFT JOIN
-> (Faculty F) ON F.FacultyID = St.MentorID GROUP BY F.FacultyID;
```

FacultyName	COUNT(St.ENO)
Aman	2
Ankit	2
Pankaj	1

3 rows in set (0.00 sec)

4. Display programme-wise total number of students taught by each faculty .

④ $\rho(T_1, \text{Student})$, $\rho(T_2, \text{Faculty})$
 $\rho(\text{Temp1, FacultyName } F, \text{count(ENO)}) (T_1 \bowtie T_2)$
 $T_1.\text{MentorID} = T_2.\text{FacultyID}$
 $\rho(\text{BranchName, Temp1.FacultyName, Temp1.Count(ENO)}) (Temp1 \bowtie \text{Branch})$

```
mysql> CREATE VIEW Temp2 AS SELECT F.FacultyName , F.BranchID , COUNT(St.ENO) AS StudCount FROM (Student st)
-> LEFT JOIN (Faculty F) On F.FacultyID = st.MentorID GROUP BY F.FacultyID ;
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> SELECT B.BranchName , Temp2.FacultyName , Temp2.StudCount FROM (Temp2) LEFT JOIN (Branch B)
-> ON Temp2.BranchID = B.BranchID ;
```

BranchName	FacultyName	StudCount
CSE	Aman	2
IT	Ankit	2
IT	Pankaj	1

3 rows in set (0.00 sec)

5. Display the names of programmes having Min. 2 faculties .

⑤ $\pi_{\text{BranchName}} \left(\text{BranchID, BranchName} \right) \left\{ \begin{array}{l} \text{Faculty} \bowtie \text{Branch} \\ \text{Count}(\text{FacultyID}) \geq 2 \end{array} \right\}$

```
mysql> SELECT BranchName From Faculty LEFT JOIN Branch On  
-> Faculty.BranchID = Branch.BranchID GROUP BY Faculty.BranchID  
-> HAVING COUNT(Faculty.FacultyID) > 1;
```

```
+-----+  
| BranchName |  
+-----+  
| CSE        |  
| IT         |  
+-----+  
2 rows in set (0.01 sec)
```

6. Display the names of programmes having Max. 5 faculties .

⑥ $\pi_{\text{BranchName}} \left(\text{BranchID, BranchName} \right) \left\{ \begin{array}{l} \text{Faculty} \bowtie \text{Branch} \\ \text{Count}(\text{FacultyID}) < 3 \end{array} \right\}$

```
mysql> SELECT BranchName From Faculty LEFT JOIN Branch On  
-> Faculty.BranchID = Branch.BranchID GROUP BY Faculty.BranchID  
-> HAVING COUNT(Faculty.FacultyID) < 3;
```

```
+-----+  
| BranchName |  
+-----+  
| CSE        |  
| IT         |  
+-----+  
2 rows in set (0.00 sec)
```

7. List Programme-wise HOD names .

⑦ $\pi_{\text{FacultyName, Role}} \left(\sigma_{\text{Role} \neq \text{"Mtr"}} (\text{Faculty} \bowtie \text{FacultyRole}) \right)$

```
mysql> SELECT F.FacultyName , FR.Role FROM (Faculty F) Left Join (FacultyRole FR)
-> ON F.FacultyID = FR.FacultyID WHERE FR.Role <> "Mtr" ;
```

FacultyName	Role
Aman	H-CSE
Rahul	H-IT
Pankaj	H-ECE

3 rows in set (0.00 sec)

8. Find the minimum, maximum, and average rank of enrolled students .

⑧ $\sigma_{\text{Student}} \left(\text{Max (QualiRank)}, \text{Min (QualiRank)}, \text{Average (QualiRank)} \right)$

```
mysql> SELECT MAX(QualiRank) , MIN(QualiRank) , AVG(QualiRank) FROM Student;
```

MAX(QualiRank)	MIN(QualiRank)	AVG(QualiRank)
68287	18211	43386.6000

1 row in set (0.03 sec)

9. Find the student intake of each programmes .

⑨ BranchName $\mathcal{F}_{\text{Count}(ENO)}$ (Student $\bowtie$ Branch)

```
mysql> SELECT B.BranchName , COUNT(S.ENO) AS StudCount FROM (Student S) LEFT JOIN (Branch B)
-> ON S.BranchID = B.BranchID GROUP BY B.BranchID;
```

BranchName	StudCount
CSE	2
IT	2
ECE	1

3 rows in set (0.00 sec)

10. List branch names having lowest student admissions .

⑩ $\mathcal{S}$ (Temp1 , BranchName $\mathcal{F}_{\text{Count}(ENO)}$ (Student $\bowtie$ Branch))

π BranchName , $\mathcal{G}_{\text{Min}(\text{Count}(ENO))}$ (Temp1)

```
mysql> CREATE VIEW Temp1 AS
-> SELECT B.BranchName , COUNT(S.Eno) AS StudCount FROM (Student S) LEFT JOIN (Branch B)
-> ON S.BranchID = B.BranchID GROUP BY B.BranchID HAVING
-> COUNT(S.Eno) = (SELECT MIN(COUNT(S.Eno)) FROM Student);
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> SELECT * FROM Temp1 WHERE StudCount = (SELECT MIN(StudCount) FROM Temp1);
```

BranchName	StudCount
ECE	1

1 row in set (0.00 sec)

Lab Exercise-6. (13/03/2024)

Q :- Optimize 5 SQL Query Using 'JavaInUse'?

SQL Query Optimizer

Database: MySQL

Input (Editable)

```
1 SELECT customer_id,count(customer_id)
2 FROM sales
3 GROUP BY customer_id
4 HAVING customer_id != '16' AND customer_id != '2';
5
```

Result

Here is the optimized version of the SQL query:

```
```sql
SELECT customer_id, COUNT(customer_id) AS customer_count
FROM sales
WHERE customer_id NOT IN ('16', '2')
GROUP BY customer_id;
```
```

Reason → This query is optimized by using the 'NOT IN' operator to filter out the rows with customer IDs '16' and '2' before the grouping, making the query more efficient. Also, the count function is renamed to 'customer_count' to make the output more comprehensible.

SQL Query Optimizer

Database: MySQL

Input (Editable)

```
1 SELECT Customers.CustomerName, Orders.OrderID
2 FROM Customers
3 LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID
4 ORDER BY Customers.CustomerName;
```

Result

The optimized version of the provided SQL query with equivalent results

```
```sql
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID
INNER JOIN (
 SELECT CustomerID
 FROM Orders
 WHERE OrderDate >= '2000-01-01'
) AS RecentOrders ON Customers.CustomerID = RecentOrders.CustomerID
ORDER BY Customers.CustomerName;
```



Scanned with OKEN Scanner

**Reason** → The outer query remains the same, retrieving customer names and order IDs using a LEFT JOIN between the 'Customers' and 'Orders' tables based on the 'CustomerID' column. The optimization involves using a subquery to filter the orders.

SQL Query Optimizer

Database: MySQL

Input (Editable)

```
1 SELECT Orders.OrderID, Employees.LastName, Employees.FirstName
2 FROM Orders
3 RIGHT JOIN Employees ON Orders.EmployeeID = Employees.EmployeeID
4 ORDER BY Orders.OrderID;
```

Result

Here is the optimized version of the provided MySQL SQL query:

```
```sql
SELECT o.OrderID, e.LastName, e.FirstName
FROM Orders AS o
RIGHT JOIN Employees AS e ON o.EmployeeID = e.EmployeeID
ORDER BY o.OrderID;
```
```

**Reason** → This version of the query uses aliases for the table names, which not only makes the query more readable but also allows us to drop the 'FROM' clause for an positional syntax for JOIN .

SQL Query Optimizer

Database: MySQL

Input (Editable)

```
1 SELECT Customers.CustomerName, Orders.OrderID
2 FROM Customers
3 CROSS JOIN Orders;
```

Result

The optimized version of the given MySQL SQL query that uses a JOIN inst

```
```sql
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```
```

**Reason** → This query performs an INNER JOIN between the "Customers" and "Orders" tables based on the common CustomerID column. It retrieves the CustomerName from the "Customers" table and OrderID from the "Orders" table for the matching rows .



The screenshot displays the SQL Query Optimizer interface. At the top, it says "SQL Query Optimizer" and "Database: MySQL". The interface is split into two main sections: "Input (Editable)" and "Result".

**Input (Editable):**

```
1 SELECT A.CustomerName AS CustomerName1, B.CustomerName AS CustomerName2, A.City
2 FROM Customers A, Customers B
3 WHERE A.CustomerID <> B.CustomerID
4 AND A.City = B.City
5 ORDER BY A.City;
```

**Result:**

Here's the optimized version of the provided MySQL SQL query:

```
```sql
SELECT DISTINCT CCC.CustomerName AS CustomerName1, CCC.City
FROM Customers A
JOIN Customers B ON A.CustomerID <> B.CustomerID AND A.City = B.City
JOIN Customers CCC ON CCC.CustomerID <> A.CustomerID AND CCC.CustomerID
ORDER BY CCC.City;
```

Reason → The 'DISTINCT' keyword is added to ensure that only unique combinations of 'CustomerName' and 'City' are considered, preventing duplicates. The query is written in a way that ensures it will return only distinct combinations of 'CustomerName' and 'City' that meet the criteria, thanks to utilizing multiple 'JOIN' operations and paying attention to the conditions under which duplicates are avoided. The 'ORDER BY' clause remains to sort the results by city.

Lab Exercise-7. (01/05/2024)

Q :- Perform 20 Easy & 10 Medium Level SQL Based Problems From HackerRank Platform ?

The screenshot shows the HackerRank user profile for Yash Pandey. The profile includes a profile picture, name, email (@pandeyyash0411), and a bio (HI ! YASH PANDEY HERE). It also displays 'My Badges' with C++ and SQL badges, 'My Certifications' (none earned), and 'Work Experience' (none added). The navigation bar at the top includes links for Prepare, Certify, Compete, and Apply, along with a search bar and user avatars.

The screenshot shows the 'Your Preparation' section for SQL. It features a progress bar indicating 43% completion (115 points to gold badge). A green button labeled 'Continue Preparation' is visible. The section is titled 'PREPARE BY TOPICS' and 'Sql'. A SQL badge icon is shown on the right.



Revising the Select Query I



Solved

Easy, SQL (Basic), Max Score: 10, Success Rate: 95.95%

Revising the Select Query II



Solved

Easy, SQL (Basic), Max Score: 10, Success Rate: 98.68%

Select All



Solved

Easy, SQL (Basic), Max Score: 10, Success Rate: 99.54%

Select By ID



Solved

Easy, SQL (Basic), Max Score: 10, Success Rate: 99.66%

Japanese Cities' Attributes



Solved

Easy, SQL (Basic), Max Score: 10, Success Rate: 99.59%

Japanese Cities' Names



Solved

Easy, SQL (Basic), Max Score: 10, Success Rate: 99.52%

Weather Observation Station 1



Solved

STATUS

- ☒ Solved
- ☐ Unsolved

SKILLS

- ☐ SQL (Basic)
- ☐ SQL (Intermediate)
- ☐ SQL (Advanced)

DIFFICULTY

- ☐ Easy
- ☐ Medium
- ☐ Hard

SUBDOMAINS

- ☐ Basic Select
- ☐ Advanced Select
- ☐ Aggregation
- ☐ Basic Join
- ☐ Advanced Join
- ☐ Alternative Queries

Easy, SQL (Basic), Max Score: 15, Success
Rate: 99.42%

Weather Observation Station 3



Solved

Easy, SQL (Basic), Max Score: 10, Success
Rate: 97.99%

Weather Observation Station 4



Solved

Easy, SQL (Basic), Max Score: 10, Success
Rate: 98.72%

Higher Than 75 Marks



Solved

Easy, SQL (Basic), Max Score: 15, Success
Rate: 98.98%

Employee Names



Solved

Easy, SQL (Basic), Max Score: 10, Success
Rate: 99.73%

Employee Salaries



Solved

Easy, SQL (Basic), Max Score: 10, Success
Rate: 99.66%

The PADS



Solved

Medium, SQL (Basic), Max Score: 30,
Success Rate: 93.57%

Occupations



Solved

Medium, Max Score: 30, Success Rate:
90.76%



New Companies

Medium, SQL (Intermediate), Max Score: 30, Success Rate: 96.09%



Solved

Revising Aggregations - The Count Function

Easy, SQL (Basic), Max Score: 10, Success Rate: 99.48%



Solved

Revising Aggregations - The Sum Function

Easy, SQL (Basic), Max Score: 10, Success Rate: 99.58%



Solved

Revising Aggregations - Averages

Easy, SQL (Basic), Max Score: 10, Success Rate: 99.77%



Solved

Average Population

Easy, SQL (Basic), Max Score: 10, Success Rate: 99.60%



Solved

Japan Population

Easy, SQL (Basic), Max Score: 10, Success Rate: 99.82%



Solved

Population Density Difference

Easy, SQL (Basic), Max Score: 10, Success Rate: 99.85%



Solved

Top Earners

Easy, SQL (Basic), Max Score: 20, Success Rate: 98.30%



Solved



Weather Observation Station 18



Solved

Medium, SQL (Basic), Max Score: 25,
Success Rate: 98.68%

Weather Observation Station 19



Solved

Medium, SQL (Basic), Max Score: 30,
Success Rate: 98.69%

African Cities



Solved

Easy, SQL (Basic), Max Score: 10, Success
Rate: 99.38%

The Report



Solved

Medium, SQL (Intermediate), Max Score:
20, Success Rate: 96.94%

Challenges



Solved

Medium, SQL (Intermediate), Max Score:
30, Success Rate: 92.21%

SQL Project Planning



Solved

Medium, SQL (Intermediate), Max Score:
40, Success Rate: 94.10%

Placements



Solved

Medium, SQL (Intermediate), Max Score:
40, Success Rate: 97.94%

Symmetric Pairs



Solved

Medium, SQL (Intermediate), Max Score:
40, Success Rate: 95.60%

